



武汉理工大学
WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发

项目名称: Social Consensus Agent
(群体协商观影 Agent)

姓名: 徐珂
学号: 2025302998
专业: 软件工程
指导教师: 戚欣

2026 年 6 月 22 日

目录

- 1 选题背景与设计思想.....1
 - 1.1 问题定义1
 - 1.2 痛点分析2
 - 1.3 核心技术与创新点2
- 2 Specs 规格文档.....5
 - 2.1 Product Spec 产品规格5
 - 2.2 Architecture Spec 架构规格6
 - 2.3 API Spec 接口规格11
 - 2.3.4 错误码体系13
- 3 系统架构与设计.....15
 - 3.1 整体架构设计15
 - 3.2 LangGraph 状态机15
 - 3.3 多智能体协作设计16
- 4 关键实现与代码展示17
 - 4.1 数据模型层17
 - 4.2 PreferenceAgent 实现.....19
 - 4.3 MediatorAgent 实现20
 - 4.4 状态机实现23
 - 4.5 LLM 适配器.....24
 - 4.6 Chroma 向量存储.....26
 - 4.7 豆瓣 MCP Server.....26
 - 4.8 Web 前端28
- 5 测试与评估29
 - 5.1 单元测试29

5.2 场景测试及截图	30
5.3 测试统计	39
6 系统升级与扩展.....	40
6.1 MVP 到现有版本扩展路线.....	40
6.2 未来扩展路线	45
6.3 AI 能力演进路径.....	47
6.4 部署架构演进	50
7 课程总结	51
7.1 遇到的问题与解决方案	51
7.2 学习收获	54
7.3 未来改进方向	55

1 选题背景与设计思想

1.1 问题定义

不是给“你”推荐，而是给“你们”推荐。Social Consensus Agent 是一个基于多智能体协作的群体观影决策系统，面向宿舍 4 人、情侣 2 人、家庭 N 人等群体场景，通过多个 Preference Agent 模拟各方代表进行协商，由 Mediator Agent 协调冲突并生成妥协方案，最终输出群体满意度最大化的观影推荐。

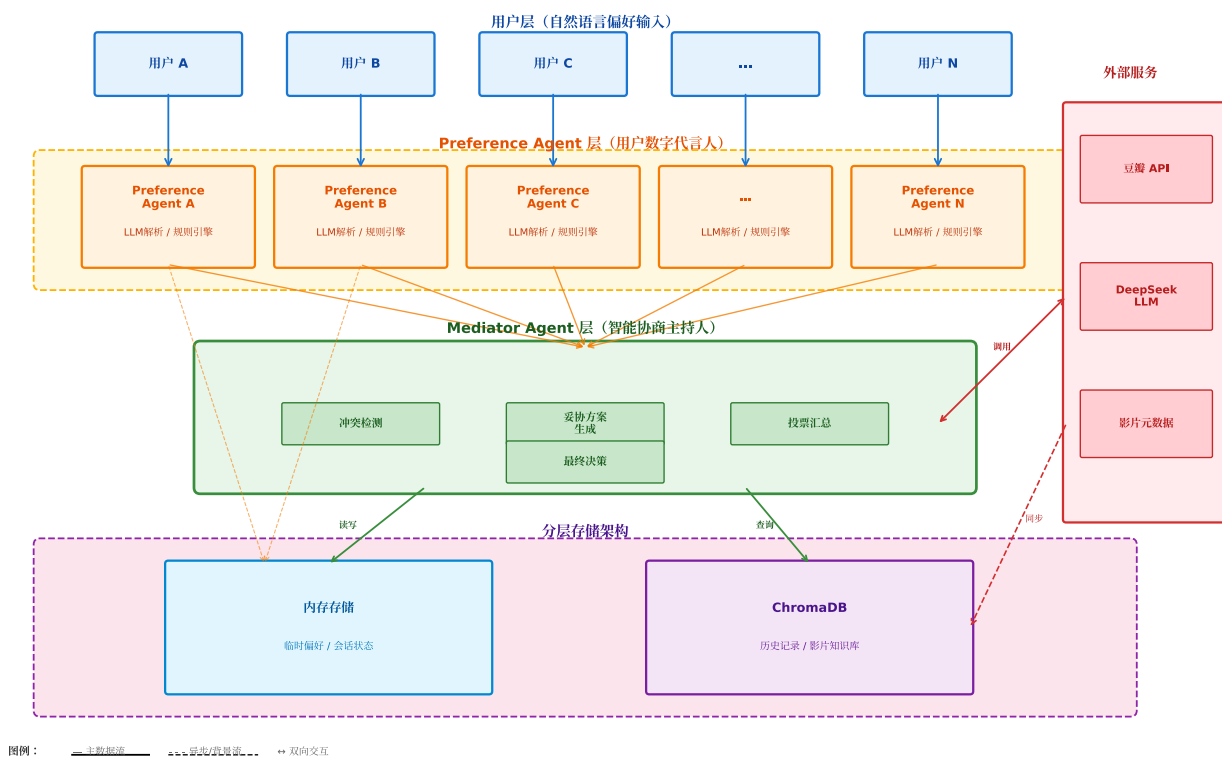


图 1.1 系统架构图

Preference Agent：作为单个用户的数字代言人，负责解析用户自然语言偏好（支持 LLM 解析和规则引擎双模式），并对提案进行投票评分；

Mediator Agent：作为智能协商主持人，负责冲突检测、妥协方案生成、投票汇总和最终决策；

分层存储架构：临时偏好使用内存存储，历史记录和影片知识库使用 ChromaDB 持久化存储。

1.2 痛点分析

多人一起看电影时面临四大痛点：

- **偏好冲突：**A 爱科幻、B 怕恐怖、C 只想看 90 分钟以内，人工协调成本高
- **协调成本高：**反复询问、比较、协商耗费大量时间
- **安全牌困境：**最终选“大家都觉得还行但谁都不满意”的中间选项
- **历史遗忘：**上次选恐怖片张三评分仅 2/10，但下次推荐时系统完全遗忘

传统推荐系统本质是单用户优化问题，面对群体场景时力不从心。本项目通过多智能体协作机制，将群体决策过程自动化、智能化，优化了多人共同观影的体验感。

技术解决方案如下：

冲突检测机制：自动识别类型冲突（如 A 偏好科幻但 B 拒绝科幻）和时长冲突（如 A 接受 90 分钟但 B 接受 180 分钟）；

妥协规则引擎：R1 类型交集规则、R2 时长最严约束、R3 评分阈值筛选、R5 安全牌兜底；

向量记忆机制：通过 ChromaDB 存储历史决策，实现“低评分影片自动降权”的进化式推荐。

1.3 核心技术与创新点

本项目的技术创新体现在以下五个方面：

多目标博弈框架：将群体推荐建模为多智能体协商问题，而非传统单用户矩阵分解，体现了多智能体协作的优势。

LLM 增强决策：通过 DeepSeek API 实现自然语言偏好解析和 nuanced 投票评分，无 API Key 时自动回退规则引擎。系统采用偏好解析和投票评分双层策略。

向量记忆机制：Chroma 数据库存储群体观影历史与满意度反馈，实现"上次低评分下次自动降权"的进化式推荐。

MCP 协议实践：标准化工具接口接入豆瓣 / IMDb API，体现企业级 Agent 的工具使用能力。

可视化前端：FastAPI + HTML5 原生前端，实时展示协商状态机流转、投票详情和协商日志。

企业级错误处理、安全防护、可观测性：错误处理方面设置了全局异常处理器捕获所有未处理异常，验证错误处理器处理 Pydantic 验证错误，还有详细的错误日志记录（包含堆栈跟踪）和友好的错误消息。

安全防护方面设置 1-10 人的用户数量限制，limit 参数范围验证（1-100）用于请求限制，还有 CORS 安全配置和 LLM/向量存储失败自动降级的异常隔离。

可观测性方面使用时间戳、级别、模块名编写结构化日志，记录所有请求的方法和路径以及每个请求的处理时间，记录错误详情和堆栈实现错误追踪。

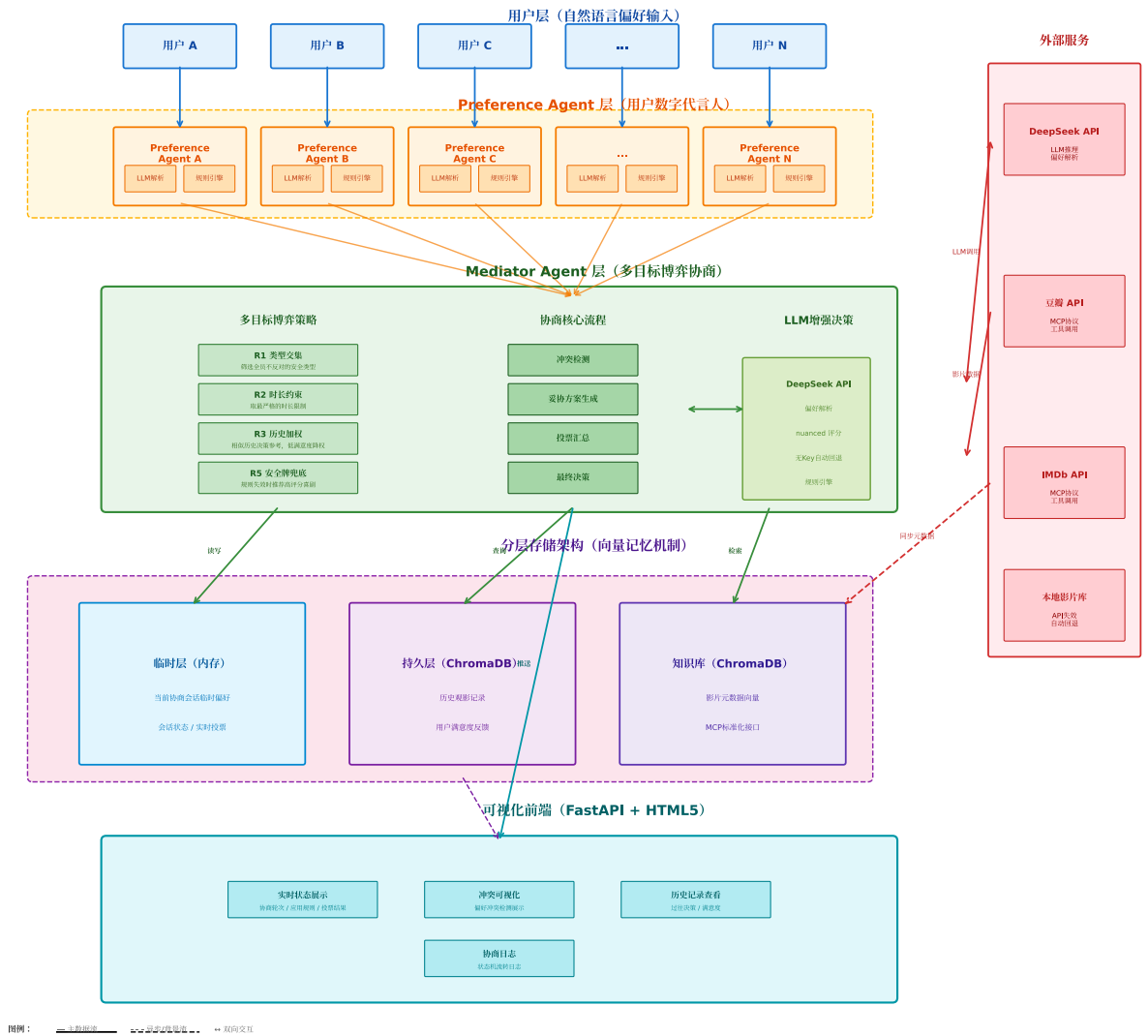


图 1.2 核心技术与创新点详细架构

2 Specs 规格文档

SDD（Spec-Driven Development，规格驱动开发）将系统规格拆分为三份独立文档，确保开发过程有据可依。

2.1 Product Spec 产品规格

Product Spec 定义系统的功能边界与业务规则。

2.1.1 协商协议

协商协议定义了 6 种消息类型：PREFERENCE（用户输入）、DECLARE（偏好声明）、PROPOSAL（提案）、VOTE（投票）、FEEDBACK（反馈）、FINAL（最终裁决）。

定义 Preference Agent 与 Mediator Agent 之间的消息类型与语义：

消息类型	发送方 -> 接收方	内容说明
PREFERENCE	User -> Pref Agent	用户的自然语言偏好描述
DECLARE	Pref Agent -> Mediator	结构化偏好声明（类型/时长/红线）
PROPOSAL	Mediator -> Pref Agent	候选影片提案及推荐理由
VOTE	Pref Agent -> Mediator	评分（1-10）+ 赞成/否决/弃权
FEEDBACK	Pref Agent -> Mediator	定性反馈（满意/不满意的原因）
FINAL	Mediator -> All	最终裁决 + 群体满意度报告

2.1.2 妥协规则

妥协规则包含 R1-R5 五条规则，按优先级依次执行：类型交叉、时长约束、历史加权、轮流优先、安全牌兜底。

Mediator Agent 在检测到冲突时，按以下优先级应用妥协规则：

规则 ID	规则名	触发条件	执行动作
R1	类型交叉	两方偏好类型不同但可交叉	推荐交叉类型影片（如科幻+动作）
R2	时长中位数	各方可接受时长范围重叠	取中位数，超时长影片降级排序
R3	历史加权	VectorDB 中有相似群体记录	低评分类型自动降权，高评分类型升权
R4	轮流优先	多方偏好完全不可交叉	本次 Alice 优先，下次 Bob（状态持久化）
R5	安全牌兜底	所有规则均无法生成可行方案	推荐豆瓣 8.0+ 喜剧片（群体接受度最高）

2.1.3 否决条件

否决条件包含 V1-V4 四项：硬约束冲突、评分低于阈值、多数否决、历史重复。

以下任一条件触发时，当前提案被否决，协商回到 resolve 状态：

否决 ID	条件名	判定标准
V1	硬约束冲突	提案触及任何用户的明确红线（如恐怖片、超过最大时长）
V2	评分低于阈值	任何 Preference Agent 对提案评分低于 3/10
V3	多数否决	超过 50% 的成员明确否决
V4	历史重复	该影片在最近 3 个月内已被群体观看过

否决条件确保少数派利益不被多数决碾压。每次否决触发后，Mediator 记录否决原因并调整下一轮提案策略。

2.2 Architecture Spec 架构规格

Architecture Spec 定义 8 个核心组件（C1-C8）的组件清单、接口矩阵和数据流规格。组件清单明确每个组件的 ID、名称、职责、依赖和生命周期。接口矩阵定义

了 C1-C8 之间的 7 条调用关系和数据契约。数据流分为偏好流、提案流和反馈流三条主线，确保数据在系统中的流转路径清晰可控。

2.2.1 组件清单

系统由以下 8 个核心组件构成，每个组件包含独立的职责、依赖与生命周期：

组件 ID	组件名称	职责	依赖	生命周期
C1	PreferenceAgent	偏好表达、利益维护、投票反馈	LLM	Session
C2	MediatorAgent	冲突检测、提案生成、最终裁决	LLM, C1	Session
C3	ConsensusGraph	LangGraph 状态机编排	C1, C2	Request
C4	CompromiseEngine	妥协规则计算引擎	C7	Request
C5	VectorStore	向量数据库（Chroma）	无	Singleton
C6	MCPManager	MCP Server 注册与路由	C7	Singleton
C7	MCP Server x3	豆瓣/IMDb/日历 API	外部 API	Singleton
C8	Logger	协商过程日志与 Trace	无	Singleton

2.2.2 接口矩阵

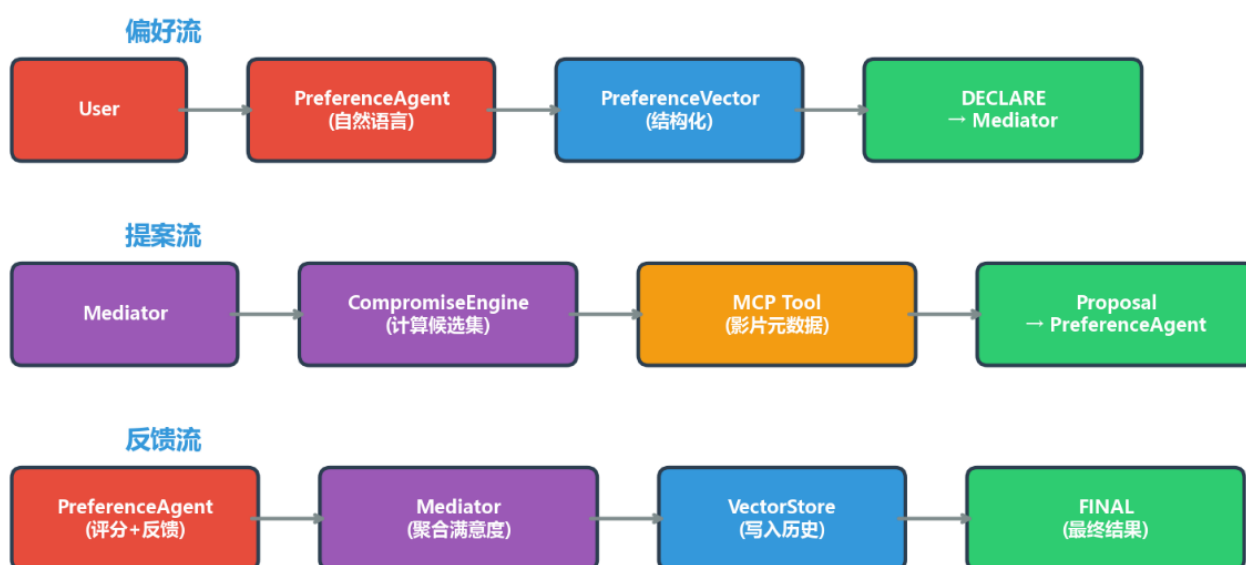
定义组件间的调用关系与数据契约（调用方向：Src -> Dst）：

Src	Dst	接口名	数据契约	调用方式
C1	C2	declare()	PreferenceVector	Message
C2	C1	send_proposal()	Proposal	Message
C1	C2	vote()	VoteResult	Message
C2	C4	resolve()	Conflict + Prefs	Function
C4	C7	search_movies()	Query Params	MCP
C2	C5	query_history()	PreferenceVector	Vector Search
C3	C8	log_state()	State + Context	Async

2.2.3 数据流规格

系统核心数据流分为三条主线：

- 偏好流：User -> PreferenceAgent（自然语言）-> 解析为 PreferenceVector（结构化）-> 通过 DECLARE 消息发送至 Mediator
- 提案流：Mediator -> 调用 CompromiseEngine 计算候选集 -> 调用 MCP Tool 获取影片元数据 -> 生成 Proposal 分发至各 PreferenceAgent
- 反馈流：PreferenceAgent -> 对 Proposal 评分 + 定性反馈 -> Mediator 聚合计算 GroupSatisfaction -> 写入 VectorStore 并输出最终结果、



图例：红色=用户/Agent | 紫色=Mediator/引擎 | 蓝色=数据 | 绿色=消息/结果 | 黄色=外部工具

图 2.1 数据流规格

2.2.4 部署拓扑

1. MVP 阶段采用单机容器化部署，架构可平滑扩展至分布式：

- 本地模式（MVP）：所有组件运行于单一 Docker Container，LLM 通过 DeepSeek API 调用，Chroma 使用内存模式
- 扩展模式（生产）：LangGraph 状态机部署为独立 Service，Chroma 升级为持久化 Server，各 MCP Server 可独立部署与水平扩展
- 降级策略：LLM API 不可用时自动切换至 Ollama 本地模型；Chroma 不可用时退化为 JSON 文件存储

2. 计划实现的最终阶段为系统的企业级生产目标架构，基于 Kubernetes 多可用区集群构建，融合服务网格与多租户隔离能力，实现全组件分布式部署与标准化运维管控。

核心部署组件：

API Gateway (Kong / Nginx)：作为集群南北向流量的唯一入口，承担请求路由转发、流量限流管控、统一身份鉴权与协议转换的核心职能。所有外部请求均通过网关接入，再按照路由规则分发至内部业务服务，同时作为安全防护的第一道边界，实现全量请求的统一管控与访问审计。

LLM 网关：构建统一大模型调度网关，集中纳管 DeepSeek、Ollama 等异构模型服务，兼容云端商用 API 与本地私有化部署模型的混合调度。网关内置智能路由策略与服务熔断机制，可根据模型实时可用性、响应延迟与调用成本动态选择最优推理路径，实现模型资源的统一管控与整体运行成本优化。

LangGraph Service 集群：系统核心业务逻辑由 LangGraph 状态机服务承载，采用 3 副本高可用（HA）模式部署于 Kubernetes 集群，通过 Deployment 控制器保障副本的稳定运行与故障自愈。依托 Pod 自动扩缩容（HPA）机制，服务可根据实时业务负载动态调整实例数量；同时支持跨可用区调度策略，避免单节点、单可用区故障导致的服务中断，保障核心业务链路的持续可用。

Chroma 集群：分向量数据库 Chroma 升级为分布式集群架构，通过 Kubernetes 持久化存储卷（PV/PVC）实现数据持久化落盘，配套定时快照与跨区域数据备份机制构建完整的数据安全体系。集群支持读写分离部署模式，在读多写少的向量检索场景下可显著提升访问吞吐量，同时具备跨区域数据容灾能力，满足大规模向量数据的存储与高并发检索需求。

MCP Server：遵循微服务解耦原则，将工具调用能力从核心业务逻辑中剥离，MCP Server 以独立微服务形态部署，支持 N 实例水平扩展。通过服务网格（Service Mesh）实现服务注册发现、流量治理与通信加密，工具服务可根据调用流量独立扩缩容、独立迭代，既降低了业务与工具模块的耦合度，也提升了工具资源的精细化利用效率。

Redis 消息队列：部署 Redis 集群作为分布式状态协调中间件，承担三大核心职能：一是全链路会话状态共享，保障多副本分布式环境下的全局状态一致性；二是异步消息队列，承载协商轮次广播等事件驱动型通信，实现组件间异步解耦；三是热点数据缓存，加速高频访问请求的响应速度，降低下游存储与服务的访问压力。

分层架构能力：

计算层：LangGraph 状态机服务以 3 副本部署于 K8s，支持 Pod 自动扩缩容与多可用区调度。

数据层：Chroma 向量数据库升级为分布式集群，配套持久化卷与定时快照，支持跨区域读写分离。

模型层：LLM 网关统一调度多模型资源，内置自动熔断与成本调度策略。

工具层：MCP Server 解耦为独立微服务，通过服务网格实现注册发现，可独立水平扩缩。

协调层：Redis 集群负责会话状态共享、异步消息队列与热数据缓存。

观测层：通过 Prometheus + Grafana 监控组件健康度，ELK 采集集群日志，Jaeger 实现分布式链路追踪。

安全层：API Gateway 提供统一鉴权，通过 Namespace 隔离多租户，以 NetworkPolicy 约束内部通信边界。

该最终阶段架构以云原生设计思想为核心，全面覆盖了企业级系统对高可用性、弹性伸缩、可观测性、安全合规的核心诉求。通过分层解耦的架构设计，系统各模块可独立演进、按需扩展，同时依托 Kubernetes 生态的标准化能力，实现了部署、运维、管控的全流程自动化，为系统在大规模生产环境下的稳定运行与持续迭代提供了坚实的架构支撑。

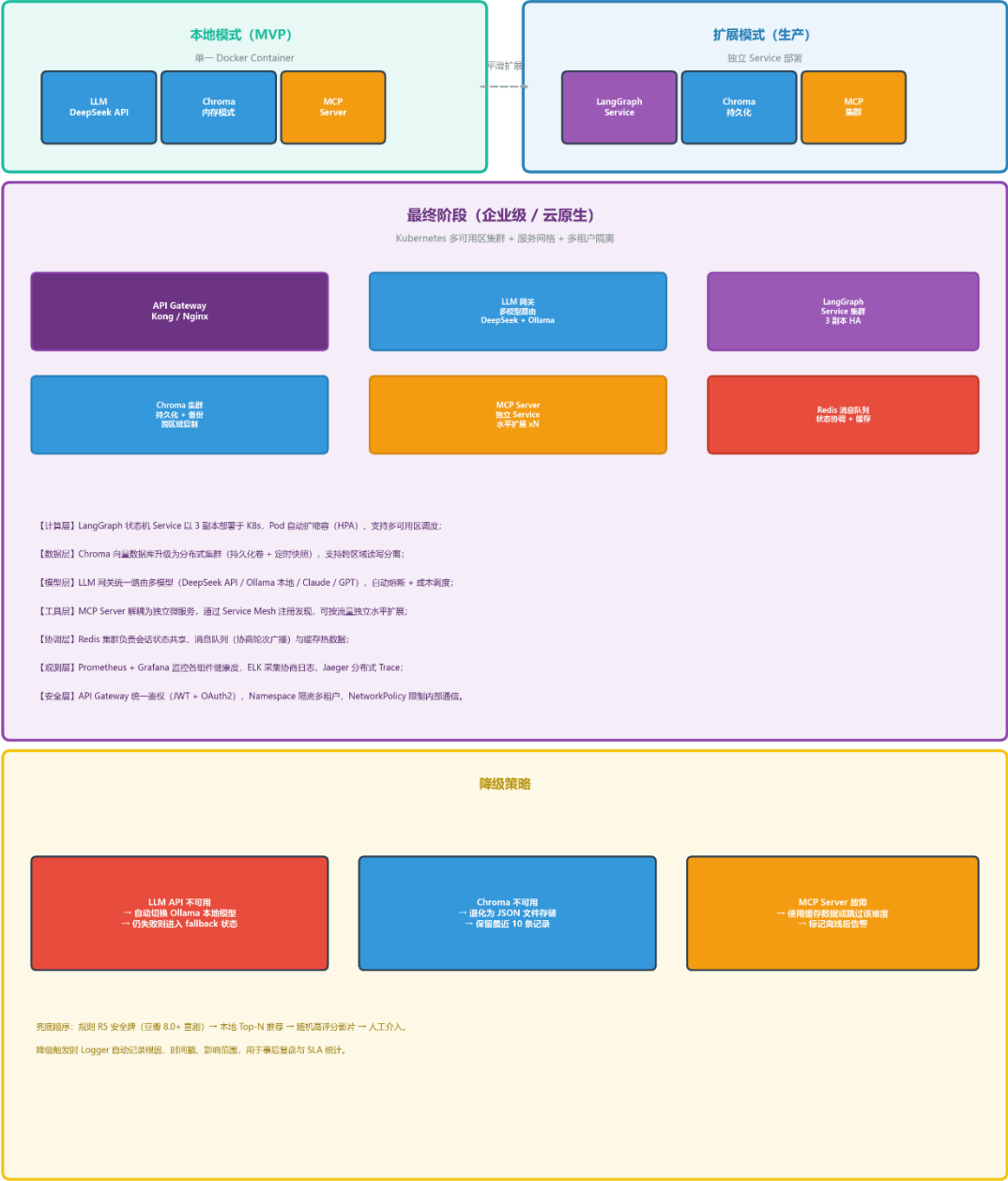


图 2.2 各阶段拓扑结构和降级策略

2.3 API Spec 接口规格

API Spec 定义 Agent 通信接口、MCP 工具接口、状态转换接口和错误码体系。

Agent 通信接口基于 Pydantic 模型, 包含 PreferenceAgent.declare()、PreferenceAgent.vote()、MediatorAgent.generate()、MediatorAgent.finalize()四个核心接口。

错误码采用 E 级分类法：E1xxx=LLM 层、E2xxx=协商层、E3xxx=工具层、E4xxx=存储层。

2.3.1 Agent 通信接口

Preference Agent 与 Mediator Agent 之间的核心通信接口（基于 Pydantic 模型）：

接口名	输入（Input）	输出（Output）
PreferenceAgent.declare()	user_text, hard_rules[], soft_prefs[]	PreferenceVector {genre, max_duration, rating_min, veto_list}
PreferenceAgent.vote()	proposal: Proposal, history[]	VoteResult {score(1-10), verdict, reason}
MediatorAgent.generate()	prefs[], round: int	Proposal {movie_id, title, genre, compromise_log}
MediatorAgent.finalize()	votes[], satisfaction: float	ConsensusResult {movie, group_score, dissenters}

2.3.2 MCP 工具接口

MCP Server 暴露的工具函数签名与返回值定义：

工具名	参数	返回	异常
douban.search	title?, genre?, rating_min?	List[MovieMeta]	E3101 超时
douban.get_detail	movie_id: str	MovieDetail	E3102 影片不存在
imdb.search	title: str, lang?	List[MovieMeta]	E3201 超时
calendar.check	duration_min, date?	{available, reason?}	E3301 日期无效

2.3.3 状态转换接口

LangGraph 状态机的节点函数签名与边条件定义：

状态节点	函数签名	出边条件	目标状态
collect_prefs	collect(state)	all_declared == true	detect_conflict
detect_conflict	detect(state)	conflict == true	resolve
detect_conflict	detect(state)	conflict == false	generate
resolve	resolve_conflict(state)	resolved == true	generate
resolve	resolve_conflict(state)	max_rounds == true	fallback

状态节点	函数签名	出边条件	目标状态
generate	propose(state)	proposals > 0	collect_votes
collect_votes	tally(state)	all_veto == false	final
collect_votes	tally(state)	any_veto == true	resolve

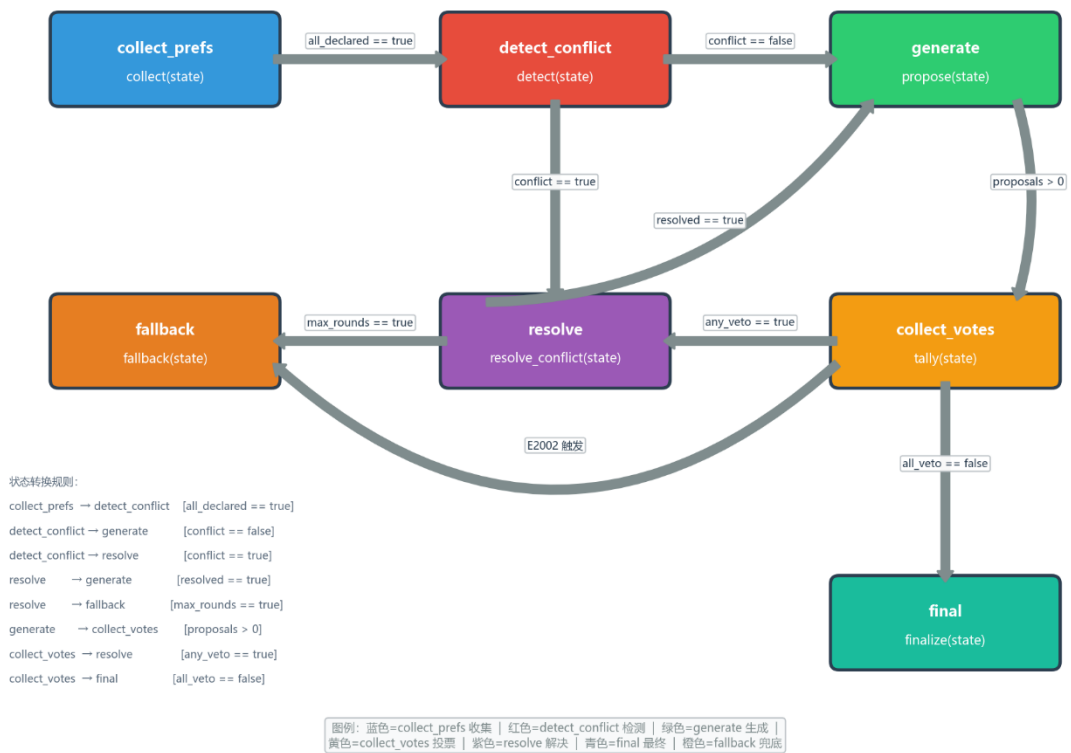


图 2.3 LangGraph 状态机示意图

2.3.4 错误码体系

系统统一错误码规范（所有接口遵循 E 级分类法）:

错误码	含义	处理策略
E1001	LLM 调用超时	自动重试 2 次，仍失败则降级至本地模型
E1002	LLM 返回格式非法	丢弃当前结果，重新生成（附带 format 约束）
E2001	协商轮次超限	强制进入 fallback 状态，推荐安全牌
E2002	所有提案被否决	触发规则 R5（安全牌兜底）
E3001	MCP Server 不可用	标记离线，使用缓存数据或跳过该维度
E3002	MCP 返回数据为空	扩大查询条件重试，仍为空则返回 E2002

错误码	含义	处理策略
E4001	向量数据库查询失败	降级为内存模式，使用最近 10 条记录

错误码分类规则：E1xxx = LLM 层错误，E2xxx = 协商层错误，E3xxx = 工具层错误，E4xxx = 存储层错误。

3 系统架构与设计

3.1 整体架构设计

系统采用分层架构设计，自上而下分为用户层、智能体层、编排层、记忆层和工具层。用户层接收群体成员的自然语言偏好输入；智能体层包含 N 个 PreferenceAgent 和 1 个 MediatorAgent；编排层通过 LangGraph 状态机驱动协商流程；记忆层使用 Chroma 向量数据库存储历史记录；工具层通过 MCP 协议接入外部 API。

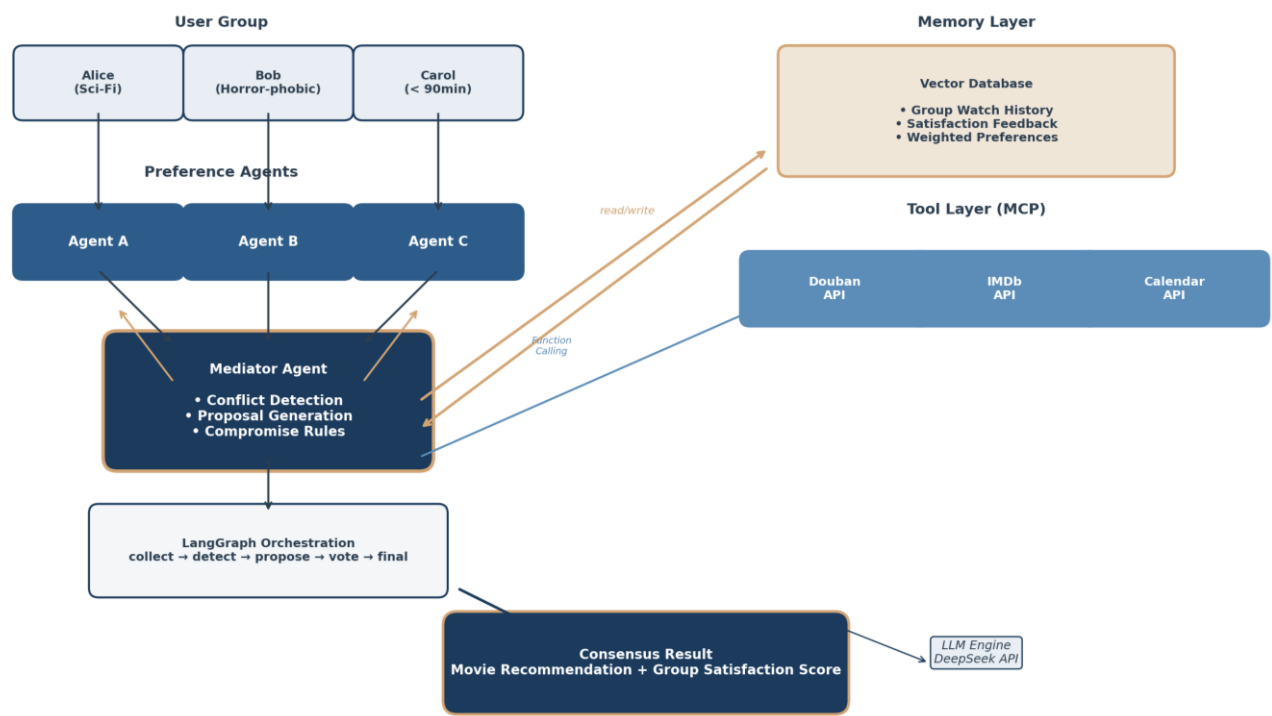


图 3.1 Social Consensus Agent 系统架构图

3.2 LangGraph 状态机

系统核心是一个由 LangGraph 编排的有限状态机，包含 6 个核心状态和 1 个兜底状态。collect_prefs 收集所有 PreferenceAgent 的偏好；detect_conflict 检测偏好冲突；resolve 应用妥协规则解决冲突；generate_proposal 基于协调后的偏好生成候选提案；collect_votes 收集各 Agent 的投票；final_decision 聚合投票产生最终结果。当超过最大解决轮次时，进入 fallback 状态执行 R5 安全牌兜底规则。

LangGraph State Machine — Consensus Workflow

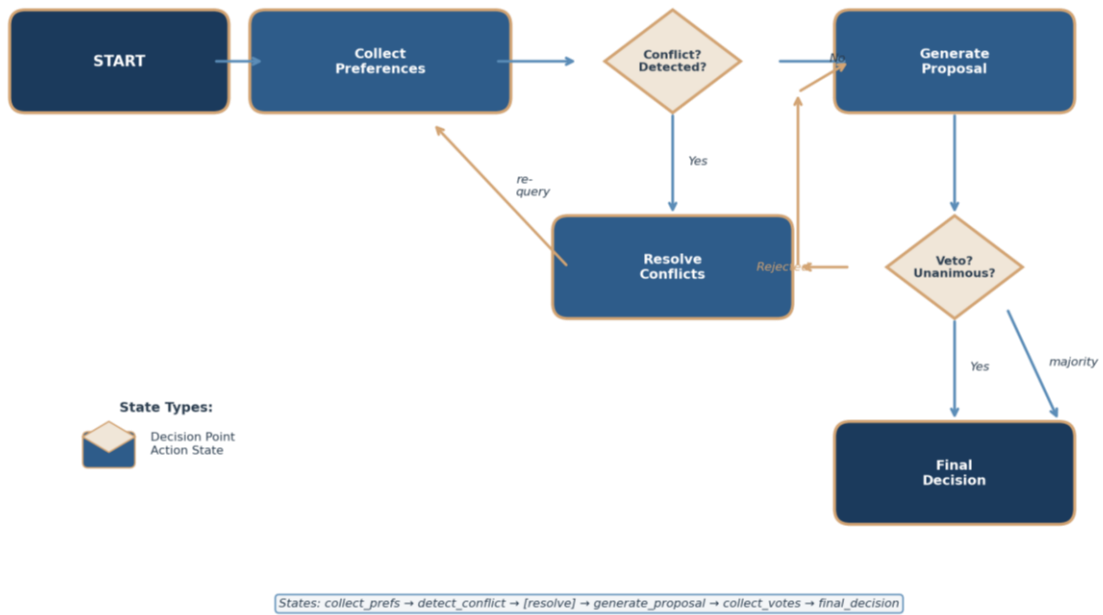


图 3.2 LangGraph 协商状态机流程图

3.3 多智能体协作设计

PreferenceAgent 是每个用户的"数字代言人", 负责将自然语言偏好解析为结构化的 PreferenceVector (包含类型偏好、时长限制、评分阈值、否决列表), 并对 Mediator 的提案进行投票评分。MediatorAgent 是协商过程的"智能会议主持人", 负责冲突检测、提案生成、议程管理和最终裁决。两者通过标准化消息协议进行通信, 确保协商过程的可控性和可观测性。

4 关键实现与代码展示

4.1 数据模型层

使用 Pydantic 定义核心数据模型，确保类型安全和数据校验。核心模型包括：

1. **PreferenceVector**（偏好向量，包含 genres、max_duration、min_rating、veto_list 等字段）。

```
1 class PreferenceVector(BaseModel):
2     user_name: str
3     genres: List[str] = Field(default_factory=list)
4     max_duration: int = Field(default=180) # 分钟
5     min_rating: float = Field(default=0.0) # 0-10
6     veto_list: List[str] = Field(default_factory=list) # 硬约束
7     soft_prefs: str = Field(default="") # 软偏好文本
```

2. **Movie**（影片元数据）。

```
01 # src/models/schemas.py:30-37
02 class Movie(BaseModel):
03     """Movie metadata."""
04     movie_id: str
05     title: str
06     genres: List[str]
07     duration: int # minutes
08     rating: float # 0-10
09     year: int = 2020
10     description: str = ""
```

3. **Proposal**（提案，包含影片和妥协规则记录）。

```
1 # src/models/schemas.py:39-44
2 class Proposal(BaseModel):
3     """Movie proposal sent by Mediator to PreferenceAgents."""
4     movie: Movie
5     compromise_rules_applied: List[str] =
6     Field(default_factory=list)
7     reason: str = ""
8     round_num: int = 1
```

4. **VoteResult**（投票结果，包含 score、verdict、reason）。

```
1 class VoteResult(BaseModel):
2     user_name: str
3     score: int = Field(ge=1, le=10) # 满意度评分 1-10
4     verdict: Verdict # APPROVE / VETO / ABSTAIN
5     reason: str # 投票理由
```

5. **ConsensusResult**（共识结果，包含最终影片、群体满意度、协商日志等）。

```
01 # src/models/schemas.py:52-62
02 class ConsensusResult(BaseModel):
03     """Final consensus output."""
04     movie: Optional[Movie] = None
05     group_score: float = 0.0
06     votes: List[VoteResult] = Field(default_factory=list)
07     negotiation_log: List[str] = Field(default_factory=list)
08     rounds_taken: int = 0
09     dissenters: List[str] = Field(default_factory=list)
10     is_fallback: bool = False
```

6. **GraphState**（LangGraph 状态对象，贯穿整个协商流程）。

```
01 @dataclass
02 class GraphState:
03     user_texts: dict[str, str]
04     preference_agents: list[PreferenceAgent]
05     mediator: MediatorAgent
06     preferences: list[PreferenceVector]
07     current_proposal: Proposal
08     votes: list[VoteResult]
09     conflicts: list[str]
10     resolve_round: int = 0
11     max_resolve_rounds: int = 3
12     rejected_movie_ids: list[str] # 存储被否决的影片 ID
13     result: ConsensusResult
14     log: list[str]
```

4.2 PreferenceAgent 实现

PreferenceAgent 实现了 declare()和 vote()两个核心方法。

1. declare()方法支持 LLM 和规则两种解析模式。

```
1 def declare(self, user_text: str = "") -> PreferenceVector:
2     # 策略: 优先使用 LLM, 失败时回退规则引擎
3     if self.llm and self.llm.is_available():
4         try:
5             return self._declare_llm(text)
6         except Exception as e:
7             print(f" [LLM parse failed, fallback to rules: {e}]")
8     return self._declare_rules(text)
```

LLM 模式下通过 DeepSeek API 调用 PREFERENCE_PARSE_PROMPT 模板进行自然语言解析:

```
1 PREFERENCE_PARSE_PROMPT = """ 你是一个观影偏好解析专家。请根据用户的自
2 然语言描述,
3 提取以下结构化信息: genres、max_duration、min_rating、veto_list...
4 只返回 JSON, 不要 markdown 代码块。 """
```

规则模式下使用关键词匹配和正则表达式提取偏好信息,

- 否定词检测: 检查类型词前方 8 字符内是否包含"不要"、"怕"、"拒绝"等否定前缀
- 时长提取: 匹配"90 分钟"、"两小时"、"不超过 X 分钟"等模式
- 评分阈值: 检测"高分"、"8 分以上"等偏好
- 特别处理了否定词过滤 (如"不要恐怖片"中的"恐怖"应加入 veto_list 而非 genres)。

2. `vote()`方法首先检查硬约束（否决列表和时长限制），然后根据类型匹配度、评分和时长偏好计算 1-10 分的满意度评分。

```
01 def vote(self, proposal: Proposal) -> VoteResult:
02     # V1: 硬约束检查（规则引擎，优先执行）
03     for veto_genre in p.veto_list:
04         if veto_genre in movie.genres:
05             return VoteResult(score=1, verdict=Verdict.VETO, ...)
06
07     # V1: 时长检查
08     if movie.duration > p.max_duration:
09         return VoteResult(score=2, verdict=Verdict.VETO, ...)
10
11     # 评分计算: LLM 模式优先，规则回退
12     return self._vote_llm(proposal) if self.llm else
13     self._vote_rules(proposal)
```

4.3 MediatorAgent 实现

MediatorAgent 实现了 `detect_conflicts()`、`generate_proposal()`、`tally_votes()`和 `finalize()`四个核心方法。

`detect_conflicts()`检测 PreferenceVector 之间的类型冲突和时长冲突。

```
01 def detect_conflicts(self, preferences: List[PreferenceVector]) ->
02     List[str]:
03     for i in range(n):
04         for j in range(i + 1, n):
05             # 类型冲突: A 偏好但 B 否决
06             for g in p1.genres:
07                 if g in p2.veto_list:
08                     conflicts.append(f"类型冲突: {p1.user_name}偏好
09 '{g}'但{p2.user_name}拒绝")
10             # 时长冲突: 超过 30 分钟差异
11             if p1.max_duration < p2.max_duration - 30:
12                 conflicts.append(f"时长冲突: {p1.user_name}接受
13 ≤{p1.max_duration}min...")
```

generate_proposal()按 R1-R3 优先级执行妥协规则：R1 过滤冲突类型保留安全交集，R2 使用最严格的时长约束，R3 查询 Chroma 历史记录进行加权。

```
01 def generate_proposal(self, preferences, round_num,
02 exclude_movie_ids):
03     # === R1: 类型交集 ===
04     safe_genres = set()
05     for p in preferences:
06         for g in p.genres:
07             # 检查该类型是否被其他成员否决
08             if not any(g in other.veto_list for other in
09 preferences):
10                 safe_genres.add(g)
11
12     # === R2: 时长最严约束 ===
13     min_max_duration = min(p.max_duration for p in preferences)
14
15     # === R3: 评分阈值 ===
16     min_rating = max(p.min_rating for p in preferences)
17
18     # === 优先使用 MCP 工具搜索 ===
19     mcp_results = self._try_mcp_search(candidate_genres, max_dur,
20 min_rating, exclude_ids)
```


tally_votes()聚合投票计算群体满意度，检测多数否决条件。

```
01 # src/agents/mediator_agent.py:190-216
02 def tally_votes(
03     self,
04     votes: List[VoteResult],
05     preferences: List[PreferenceVector]
06 ) -> ConsensusResult:
07     """Aggregate votes and produce result."""
08     total = len(votes)
09     vetoes = [v for v in votes if v.verdict == Verdict.VETO]
10
11     # 多数否决判定: 超过半数否决则提案失败
12     if len(vetoes) > total / 2:
13         return ConsensusResult(
14             movie=None,
15             group_score=0.0,
16             votes=votes,
17             negotiation_log=[f"V3-多数否决: {len(vetoes)}/{total}
18 成员否决"],
19             dissenters=[v.user_name for v in vetoes]
20         )
21
22     # 计算群体满意度: 所有投票评分的平均值
23     group_score = sum(v.score for v in votes) / total if total > 0
24 else 0
25
26     # 识别异议者: 否决或评分低于5分的用户
27     dissenters = [v.user_name for v in votes if v.verdict ==
28 Verdict.VETO or v.score < 5]
29
30     return ConsensusResult(
31         movie=None,
32         group_score=group_score,
33         votes=votes,
34         dissenters=dissenters
35     )
```

finalize()生成最终结果并写入向量数据库。

```
1 def finalize(self, proposal: Proposal, votes: List[VoteResult]) ->
2 ConsensusResult:
3     result = self.tally_votes(votes, [])
4     result.movie = proposal.movie
5     result.rounds_taken = proposal.round_num
6
7     # 存储到向量数据库
8     if self.vector_store:
9         self.vector_store.add_watch_record(result, preferences)
10    return result
```

4.4 状态机实现

使用 dataclass 定义 GraphState 状态对象，包含用户输入、Agent 实例、偏好集合、当前提案、投票结果、冲突记录和协商日志等字段。7 个节点函数（node_*）分别对应 7 个状态，每个函数接收当前状态并返回下一个状态。STATE_MAP 字典将状态名映射到节点函数，构建了一个清晰的有限状态机。状态转换支持循环（collect_votes 检测到否决时回到 resolve 重新协商）和终止条件（达到最大轮次进入 fallback）。

状态节点	功能	转换逻辑
node_collect_prefs	收集用户偏好	→ detect_conflict
node_detect_conflict	检测偏好冲突	有冲突→resolve， 无冲突→generate_proposal
node_resolve	冲突解决轮次	超过最大轮次→fallback， 否则→generate_proposal
node_generate_proposal	生成妥协提案	→ collect_votes
node_collect_votes	收集投票	有否决且未超轮次→resolve， 否则→final_decision
node_final_decision	最终裁决	终止状态
node_fallback	安全牌兜底	终止状态

4.5 LLM 适配器

采用适配器模式设计 LLM 层。

```
01 class BaseLLM(ABC):
02     @abstractmethod
03     def chat(self, system_prompt: str, user_prompt: str) ->
04     LLMResponse: ...
05
06     @abstractmethod
07     def is_available(self) -> bool: ...
08
09 class DeepSeekLLM(BaseLLM):
10     def __init__(self, config: LLMConfig | None = None):
11         self.config.provider = "deepseek"
12         self.config.base_url = "https://api.deepseek.com/v1"
13         self.config.model = "deepseek-chat"
14         self._client = OpenAI(api_key=..., base_url=...,
15 timeout=30)
16
17     def chat(self, system_prompt: str, user_prompt: str) ->
18     LLMResponse:
19         # 支持重试逻辑 (最多2次)
20         for attempt in range(3):
21             try:
22                 resp = self._client.chat.completions.create(...)
23                 return
24             LLMResponse(content=resp.choices[0].message.content, ...)
25             except Exception as e:
26                 if attempt < 2: continue
27                 return LLMResponse(error=str(e))
28
29 class MockLLM(BaseLLM):
30     # 规则引擎回退, 保持与 LLM 相同的输出格式
31     def chat(self, system_prompt: str, user_prompt: str) ->
32     LLMResponse:
33         if "观影偏好解析" in system_prompt:
34             return
35         LLMResponse(content=self._parse_preference(user_prompt), ...)
36         elif "观影评价" in system_prompt:
37             return
38         LLMResponse(content=self._vote_on_movie(user_prompt), ...)
```

BaseLLM 抽象基类定义统一接口（chat()和 is_available()）。DeepSeekLLM 通过 OpenAI-compatible API 调用 DeepSeek 模型，支持自动重试和错误处理。MockLLM 在 API Key 不可用时提供规则引擎回退，确保系统始终可运行。create_llm()工厂函数自动检测可用性，优先使用 DeepSeek。Prompt 模板分离了偏好解析和投票评价两种任务，便于后续优化。

```
1 def create_llm(config: LLMConfig | None = None) -> BaseLLM:
2     ds = DeepSeekLLM.from_env()
3     if ds.is_available():
4         print(f" [LLM] Using DeepSeek provider
5 (model={ds.config.model})")
6         return ds
7     print(f" [LLM] DeepSeek unavailable, using Mock (rule-
8 based)")
9     return MockLLM(config)
```

4.6 Chroma 向量存储

分层存储架构：

```
1 HierarchicalVectorStore
2 |— ChromaDB (持久化)
3 |   |— 观影历史记录 (watch_history collection)
4 |   |— cosine 相似度检索
5 |— Memory (临时)
6     |— 当前会话偏好
```

WatchRecord 类将群体观影记录转换为文本表示，用于向量嵌入。

```
01 class WatchRecord:
02     def to_text(self) -> str:
03         return f"影片《{self.title}》类型{'/'.join(self.genres)}时长
04 {self.duration}分钟 \
05         群体满意度{self.group_score}分 用户偏好:
06 {pref_texts}"
07
08 class ChromaVectorStore:
09     def query_similar(self, preferences, n_results=3) ->
10 List[dict]:
11         # cosine 相似度检索
12         results = self._collection.query(
13             query_texts=[query_text],
14             n_results=n_results,
15         )
16         # 返回相似度加权的检索结果
```

ChromaVectorStore 使用 cosine 相似度进行语义检索，在生成提案前自动查询相似历史决策作为参考。MemoryVectorStore 提供关键词匹配的回退方案，确保 Chroma 未安装时系统仍可运行。create_vector_store() 工厂函数自动检测 Chroma 可用性并选择合适的实现。

4.7 豆瓣 MCP Server

douban_mcp.py 实现了 MCP 协议的核心功能：search 工具支持按标题、类型、评分、时长搜索影片，detail 工具支持按 ID 获取影片详情。MCP_TOOLS 字典注册所有可用工具，call_tool() 提供统一调用接口。当 DOUBAN_API_KEY 环境变量未设置时，自动回退到本地 50 部影片数据库，确保 MVP 阶段无需外部 API 即可运行。

MCP 协议实现，工具注册与调用接口：

```
01 MCP_TOOLS = {
02     "douban.search": {
03         "function": search,
04         "description": "Search movies by title, genre, rating,
05 duration",
06         "parameters": {...}
07     },
08     "douban.detail": {
09         "function": get_detail,
10         "description": "Get movie details by ID",
11         "parameters": {...}
12     },
13 }
14
15 def call_tool(tool_name: str, **kwargs) -> str:
16     """统一工具调用接口"""
17     tool = MCP_TOOLS[tool_name]
18     return tool["function"](**kwargs)
```

搜索实现：

优先爬取豆瓣电影页面，解析 window.__DATA__ 内联 JSON

支持类型、评分、时长等多维度过滤

API 不可用时自动回退到本地 50 部影片数据库

4.8 Web 前端

前端采用原生 HTML5 + Tailwind CSS CDN 实现，零构建步骤。

```
01 <!-- 动态用户输入 -->
02 <div id="user-inputs" class="space-y-4"></div>
03 <button onclick="addUser()" class="btn-primary">+ 添加成员</button>
04
05 <!-- 状态机可视化 -->
06 <div id="state-indicator" class="flex items-center space-x-4">
07   <div id="state-collect_prefs" class="state-dot"></div>
08   <span>收集偏好</span>
09 </div>
10
11 <!-- 推荐结果卡片 -->
12 <div id="result-card" class="hidden">
13   <h3 id="movie-title"></h3>
14   <div id="vote-details"></div>
15 </div>
```

前端 API 调用：

```
01 const res = await fetch(API_BASE + '/api/negotiate', {
02   method: 'POST',
03   headers: {'Content-Type': 'application/json'},
04   body: JSON.stringify({users: [...], use_llm: true, use_chroma:
05 true})
06 });
07
08 // 请求超时处理
09 const controller = new AbortController();
10 const timeoutId = setTimeout(() => controller.abort(), 60000);
11 const res = await fetch(url, {signal: controller.signal});
```

核心功能包括：动态用户输入（添加/删除成员）、5 个预设场景一键加载、6 状态协商状态机实时高亮、推荐结果卡片（影片信息+满意度）、投票详情（评分+理由+赞成/否决/弃权标识）、协商日志（按类型着色）。前端通过 Fetch API 调用 FastAPI 后端，CORS 已配置允许跨域访问。

5 测试与评估

5.1 单元测试

test_agents.py 包含 15 个单元测试，覆盖 PreferenceAgent 的偏好解析（类型提取、否定词过滤、时长解析、否决列表）和投票逻辑（赞成、否决类型、否决时长、高分影片），以及 MediatorAgent 的冲突检测、提案生成、投票计票和安全牌兜底。

PreferenceAgent 测试（9 个）:

测试方法	测试内容	验证点
test_declare_parses_genres	类型提取	科幻类型正确识别，否定类型不进入 genres
test_declare_parses_veto	否决列表解析	"不要恐怖片" → 恐怖进入 veto_list
test_declare_parses_duration	时长解析	"90 分钟以内" → max_duration ≤ 95
test_declare_handles_negation_combined	组合否定词	"拒绝恐怖和战争" → 两者都进入 veto_list
test_declare_default_genre	默认类型	无类型指定时默认 "剧情"
test_vote_approve	赞成投票	匹配影片返回 APPROVE，评分≥6
test_vote_veto_genre	类型否决	触及否决类型返回 VETO
test_vote_veto_duration	时长否决	超时时长返回 VETO
test_vote_high_score	高分影片	完美匹配评分≥8

MediatorAgent 测试（6 个）:

测试方法	测试内容	验证点
test_detect_conflict_genre_clash	类型冲突检测	A 偏好科幻但 B 否决科幻
test_detect_conflict_no_conflict	无冲突检测	双方偏好喜剧，无冲突
test_generate_proposal_returns_valid	提案生成	返回有效影片，时长≤约束+5
test_generate_proposal_respects_veto	否决约束	提案不包含被否决类型
test_generate_proposal_none_for_impossible	无解场景	极端约束返回 None
test_tally_votes_calculates_score	计票逻辑	$(8+6)/2 = 7.0$ 正确计算
test_tally_votes_majority_veto	多数否决	3 人中 2 人否决 → 群体满意度为 0

测试方法	测试内容	验证点
test_safe_fallback	安全牌兜底	返回高评分喜剧片 (≥8.0)

test_graph.py 包含 17 个状态机测试，覆盖 6 个状态的转换逻辑、全 workflow 闭环、最大轮次限制、fallback 触发和边界条件。使用 make_state() 辅助函数构建测试状态，确保每个节点函数的独立可测性。

测试类	测试内容	数量
TestCollectPrefs	偏好收集节点	1
TestDetectConflict	冲突检测节点	2
TestResolve	冲突解决节点	2
TestGenerateProposal	提案生成节点	2
TestCollectVotes	投票收集节点	2
TestFinalDecision	最终裁决节点	1
TestFallback	安全牌兜底节点	1
TestFullWorkflow	完整 workflow	6

关键辅助函数：

```

1 def make_state(user_texts: dict[str, str]) -> GraphState:
2     """辅助函数：构建测试状态，确保节点函数独立可测"""
3     agents = [PreferenceAgent(name, text) for name, text in
4 user_texts.items()]
5     return GraphState(
6         user_texts=user_texts,
7         preference_agents=agents,
8         mediator=MediatorAgent(),
9     )

```

5.2 场景测试及截图

test_scenarios.py 包含 17 个集成测试，覆盖 5 个 Demo 场景和多个边界用例：

场景 1： 科幻 vs 喜剧（2 人，验证类型交叉和时间约束）

```

1 state = build_consensus_workflow({
2     "Alice": "我喜欢科幻片，时长不要超过 120 分钟，不要恐怖片",
3     "Bob": "我想看喜剧片，轻松一点的，时长 90 分钟以内最好",
4 })

```

- 验证类型交叉和时间约束
- 结果不应包含恐怖片（Alice 否决）
- 时长不应超过 95 分钟（Bob 的约束）

结果：

1. Mock LLM

2. DeepSeek API & 豆瓣 API 接入



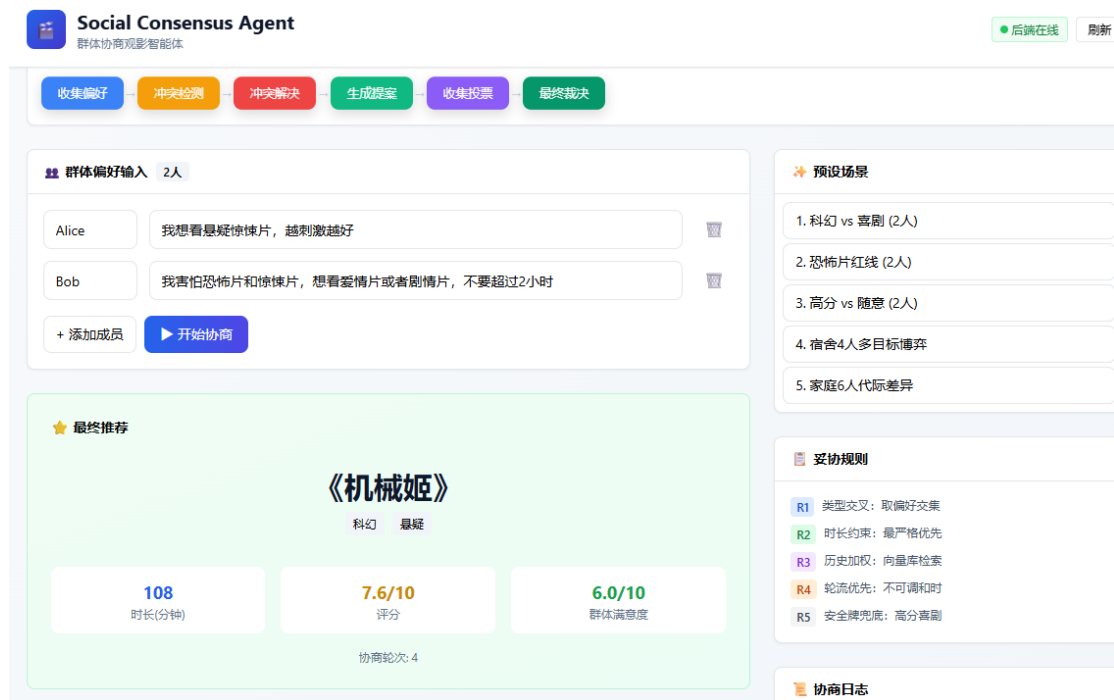
场景 2：恐怖片红线（验证否决机制）

```
1 state = build_consensus_workflow({
2     "Alice": "我想看悬疑惊悚片，越刺激越好",
3     "Bob": "我害怕恐怖片和惊悚片，想看爱情片或者剧情片",
4 })
```

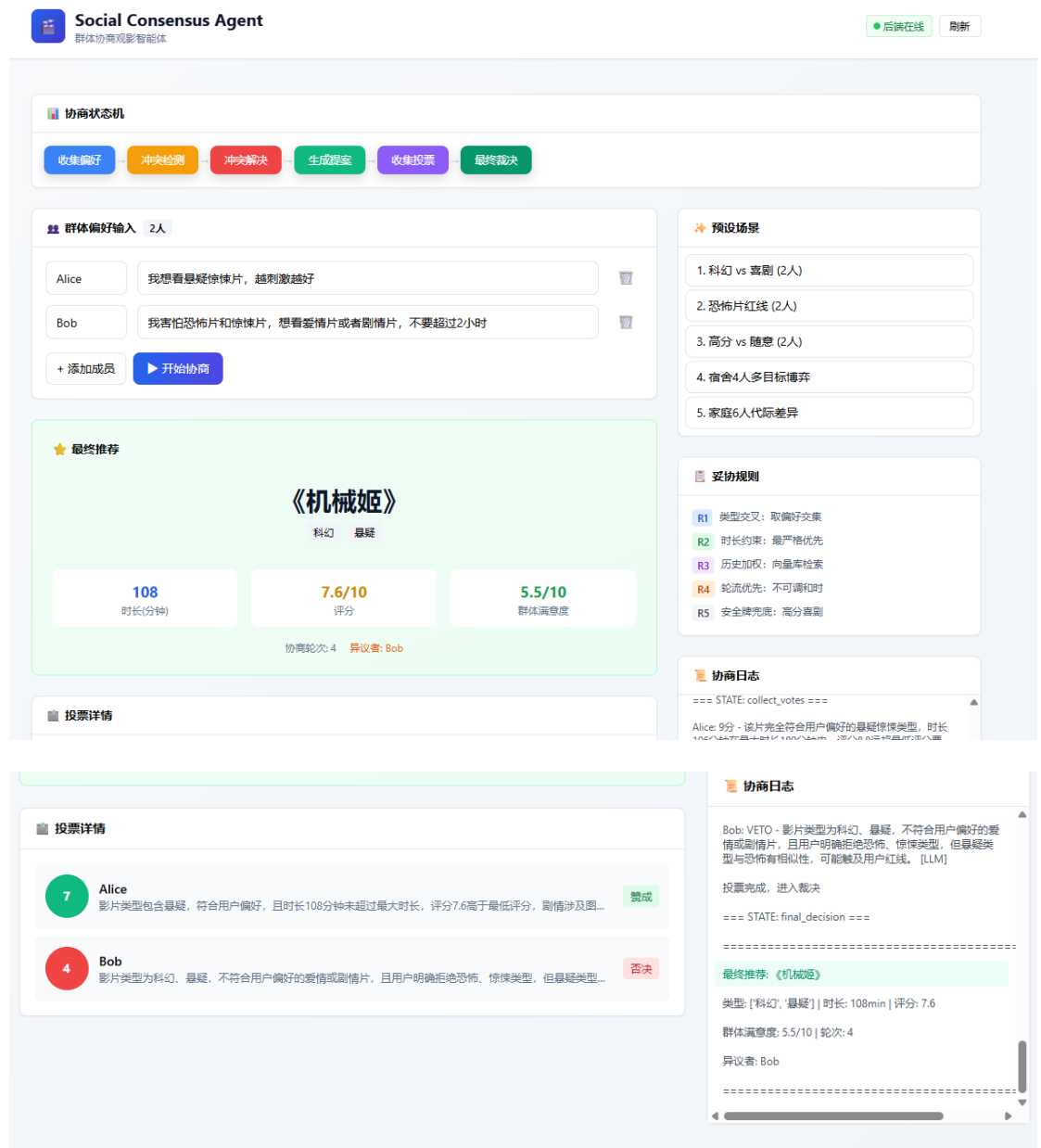
- 验证否决机制
- Bob 否决恐怖/惊悚，结果应安全

结果：

1. Mock LLM



2. DeepSeek API & 豆瓣 API 接入



场景 3: 高分 vs 随意（验证评分敏感）

```
1 state = build_consensus_workflow({
2     "Alice": "我只看高分电影，评分至少 8 分以上",
3     "Bob": "我随便，什么都行，搞笑的最好",
4 })
```

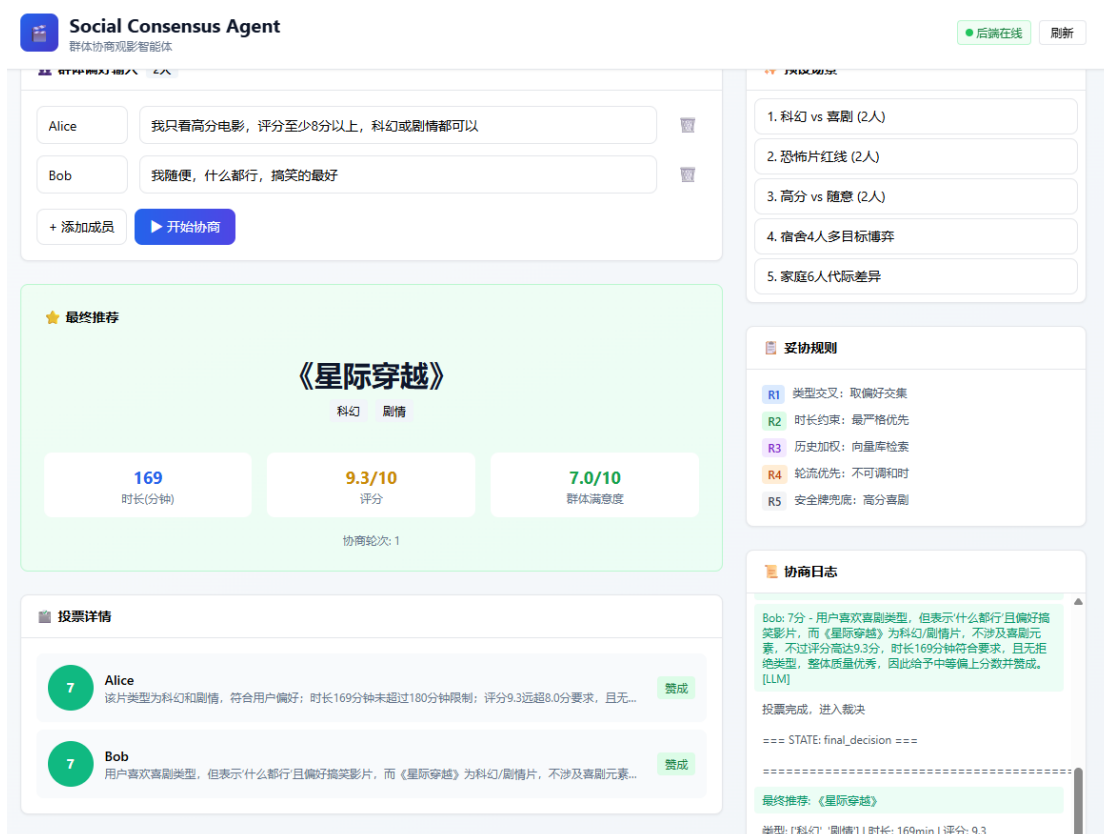
- 验证评分敏感性
- 结果评分 ≥ 8.0

结果:

1. Mock LLM



2. DeepSeek API & 豆瓣 API 接入



场景 4：宿舍 4 人多目标博弈（验证 N 人扩展）

```

1 state = build_consensus_workflow({
2     "Alice": "我喜欢科幻和动作片，评分高的",
3     "Bob": "我只看喜剧和动画片，超过 100 分钟的不行",
4     "Carol": "我要看爱情片或者剧情片，拒绝恐怖和战争",
5     "David": "我都可以，最好是大家都喜欢的，评分 8 分以上",
6 })

```

- 验证 N 人扩展
- 4 人均参与投票，满意度 > 0

结果：

1. Mock LLM

Social Consensus Agent
群体协商观影智能体

● 后跳在线 刷新

收集偏好 冲突检测 冲突解决 生成提案 收集投票 最终裁决

群体偏好输入 4人

成员	偏好描述
Alice	我喜欢科幻和动作片，评分高的，不要太长
Bob	我只看喜剧和动画片，超过100分钟的不行
Carol	我要看爱情片或者剧情片，拒绝恐怖和战争
David	我都可以，最好是大家都喜欢的，评分8分以上

+ 添加成员 开始协商

最终推荐

《干与干寻》
动画 奇幻 冒险

指标	值
时长(分钟)	125
评分	9.4/10
群体满意度	7.5/10

协商轮次: 1

投票详情

预设场景

1. 科幻 vs 喜剧 (2人)
2. 恐怖片红线 (2人)
3. 高分 vs 随意 (2人)
4. 宿舍4人多目标博弈
5. 家庭6人代际差异

妥协规则

- R1 类型交叉：取偏好交集
- R2 时长约束：最严格优先
- R3 历史加权：向量库检索
- R4 轮流优先：不可调和时
- R5 安全牌兜底：高分喜剧

协商日志

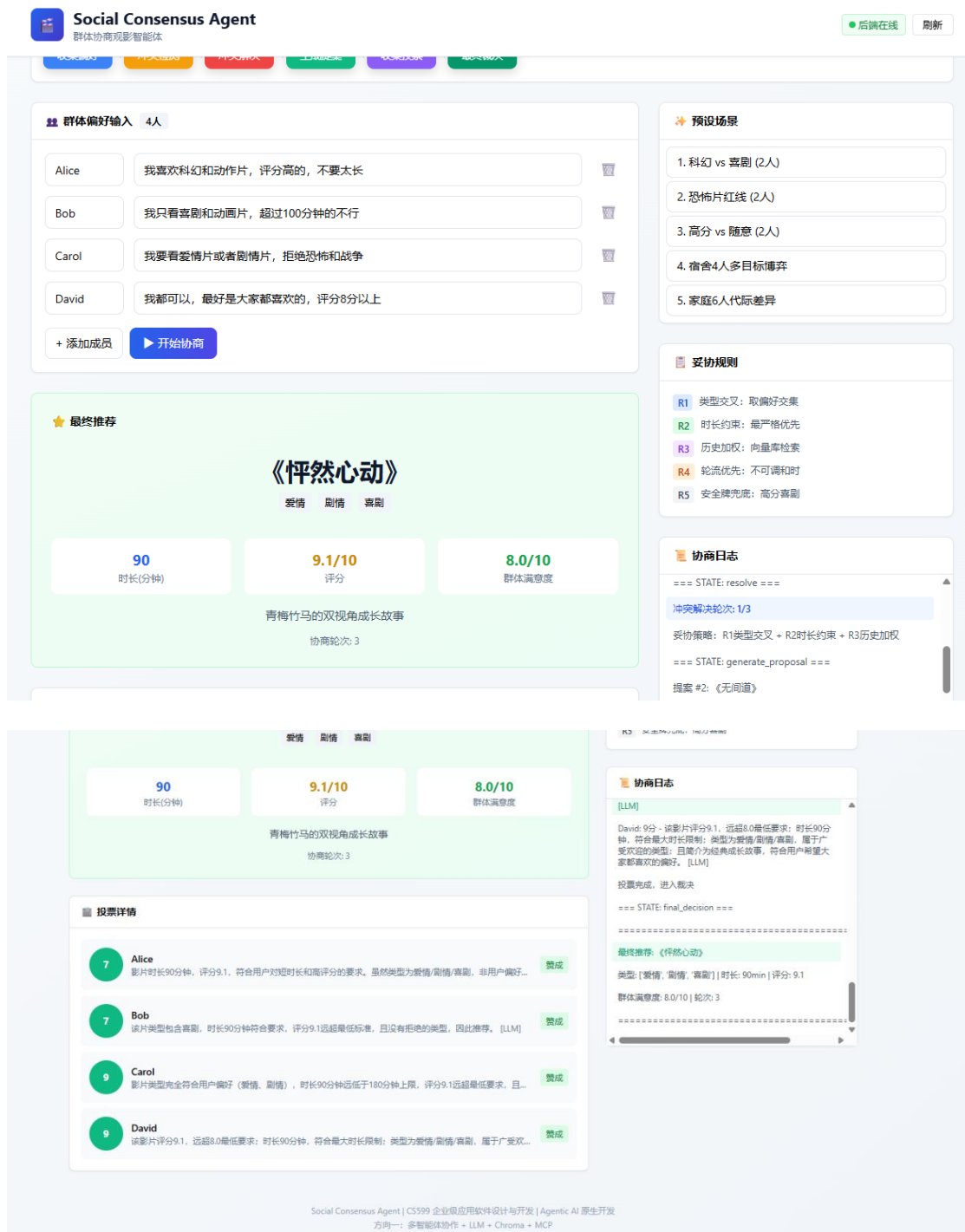
```

bob: 偏好=[喜剧, 动画], 最大时长=180min, 最低评分=0.0, 否决=[]
Carol: 偏好=[爱情, 剧情], 最大时长=180min, 最低评分=0.0, 否决=[恐怖, 战争]
David: 偏好=[剧情], 最大时长=180min, 最低评分=8.0, 否决=[]

=== STATE: detect conflict ===

```

2. DeepSeek API & 豆瓣 API 接入

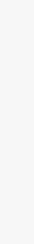


场景 5：家庭 6 人代际差异（验证大规模群体）

```

1 state = build_consensus_workflow({
2     "爷爷": "我想看战争片或者历史剧情片",
3     "奶奶": "爱情片或者家庭剧都可以",
4     "爸爸": "动作片或者悬疑片",
5     "妈妈": "喜剧片最好",
6     "哥哥": "科幻大片，越长越好",
7     "妹妹": "动画片或者爱情片，不要超过 2 小时",
8

```



- 结果:

1. Mock LLM

Social Consensus Agent

群体协商观影智能体

后验在线

刷新

收集偏好

冲突检测

冲突解决

生成提案

收集投票

最终裁决

群体偏好输入

6人

爷爷

我想看战争片或者历史剧情片

奶奶

爱情片或者家庭剧都可以，不要太悲伤

爸爸

动作片或者悬疑片，刺激一点的

妈妈

喜剧片最好，全家人一起笑

哥哥

科幻大片，特效好的，越长越好

妹妹

动画片或者爱情片，不要超过2小时

+ 添加成员

▶ 开始协商

最终推荐

《美丽人生》

剧情 喜剧 战争

116

时长(分钟)

9.5/10

评分

7.8/10

群体满意度

协商轮次: 1

预设场景

1. 科幻 vs 喜剧 (2人)

2. 恐怖片红线 (2人)

3. 高分 vs 随意 (2人)

4. 宿舍4人多目标博弈

5. 家庭6人代际差异

妥协规则

R1 类型交叉: 取偏好交集

R2 时长约束: 最严格优先

R3 历史加权: 向量库检索

R4 轮流优先: 不可调和时

R5 安全牌兜底: 高分喜剧

协商日志

=== STATE: collect_prefs ===

爷爷: 偏好=['剧情', '战争'], 最大时长=180min, 最低评分=0.0, 否决=[]

奶奶: 偏好=['爱情'], 最大时长=180min, 最低评分=0.0, 否决=[]

爸爸: 偏好=['动作', '悬疑'], 最大时长=180min, 最低评分=0.0, 否决=[]

2. DeepSeek API & 豆瓣 API 接入

Social Consensus Agent

群体协商观影智能体

后端在线

刷新

收集偏好

冲突检测

冲突解决

生成提案

收集投票

最终裁决

群体偏好输入6人

爷爷我想看战争片或者历史剧情片

奶奶爱情片或者家庭剧都可以，不要太悲伤

爸爸动作片或者悬疑片，刺激一点的

妈妈喜剧片最好，全家人一起笑

哥哥科幻大片，特效好的，越长越好

妹妹动画片或者爱情片，不要超过2小时

+ 添加成员

开始协商

最终推荐

《美丽人生》

剧情 喜剧 战争

116

时长(分钟)

9.5/10

评分

6.8/10

群体满意度

预设场景

1. 科幻 vs 喜剧 (2人)

2. 恐怖片红线 (2人)

3. 高分 vs 随意 (2人)

4. 宿舍4人多目标博弈

5. 家庭6人代际差异

妥协规则

R1 类型交叉：取偏好交集

R2 时长约束：最严格优先

R3 历史加权：向量库检索

R4 轮流优先：不可调和时

R5 安全牌兜底：高分喜剧

协商日志

妹妹: 7分 - 影片时长116分钟符合120分钟以内要求, 评分9.5远超最低标准, 且类型包含剧情和喜剧, 虽非纯动画或爱情片, 但战争背景下的父爱故事情感动人, 可能吸引喜欢爱情元素的用户。 [LLM]

投票完成, 进入裁决

=== STATE: final_decision ===

Social Consensus Agent

群体协商观影智能体

后端在线

刷新

剧情 喜剧 战争

116

时长(分钟)

9.5/10

评分

6.8/10

群体满意度

父亲用谎言保护儿子度过集中营

协商轮次: 2 异议者: 哥哥

投票详情

9 爷爷

影片类型包含战争元素, 符合用户偏好; 时长116分钟在180分钟限制内; 评分9.5远超最低要求; 故事背景... 赞成

7 奶奶

影片虽然涉及战争和集中营的悲伤背景, 但核心是父亲用爱与谎言保护孩子的家庭温情, 符合用户喜欢的... 赞成

6 爸爸

影片评分极高 (9.5), 时长116分钟在限制内, 类型虽非用户偏好的动作或悬疑, 但战争背景和紧张情节... 赞成

9 妈妈

该片为喜剧类型, 时长116分钟 (符合最大180分钟要求), 评分9.5远超最低0.0分, 且无拒绝类型。影片... 赞成

3 哥哥

影片类型为剧情、喜剧和战争, 不符合用户偏好的科幻类型; 时长116分钟较短, 且缺乏用户要求的科幻大... 弃权

7 妹妹

影片时长116分钟符合120分钟以内要求, 评分9.5远超最低标准, 且类型包含剧情和喜剧, 虽非纯动画或爱... 赞成

9.5远超最低标准, 且类型包含剧情和喜剧, 虽非纯动画或爱情片, 但战争背景下的父爱故事情感动人, 可能吸引喜欢爱情元素的用户。 [LLM]

投票完成, 进入裁决

=== STATE: final_decision ===

最终推荐: 《美丽人生》

类型: ['剧情', '喜剧', '战争'] | 时长: 116min | 评分: 9.5

群体满意度: 6.8/10 | 轮次: 2

异议者: 哥哥

边界为单人模式、空偏好、全相同偏好、极端时长约束，边界用例测试如下：

38

用例	测试内容	预期结果
test_single_user	单人模式	正常返回推荐
test_empty_preference	空偏好 ("随便")	默认推荐剧情片
test_all_same_preference	全员相同偏好	返回该类型影片
test_extreme_duration_constraint	极端约束 (30 分钟内)	触发安全牌兜底

5.3 测试统计

测试文件	测试数量	通过率	覆盖范围
test_agents.py	15	100%	Agent 核心逻辑 (偏好解析、投票、冲突检测、提案生成)
test_graph.py	17	100%	状态机转换 (7 个节点 + 完整工作流)
test_scenarios.py	17	100%	集成场景 (5 个 Demo 场景 + 4 个边界用例)
合计	49	100%	全量覆盖

测试运行方式:

- 1 | `python -m pytest tests/ -v`
- 独立可测：每个节点函数独立测试，通过 `make_state()` 构建隔离环境
 - 边界覆盖：包含极端约束、空输入、单人模式等边界场景
 - 全链路验证：从单元测试到集成测试，覆盖完整协商流程

6 系统升级与扩展

6.1 MVP 到现有版本扩展路线

6.1.1 接入真实 LLM

从 MVP 的规则引擎升级到 DeepSeek API 驱动的 LLM 增强决策。适配器模式（Adapter Pattern）确保系统可在 LLM 和规则引擎之间无缝切换。

架构设计：

```
01 # src/llm/mock.py:207-220
02 def create_llm(config: LLMConfig | None = None) -> BaseLLM:
03     """
04     工厂函数自动检测并创建 LLM provider
05     优先级: DeepSeek (有 API Key) > Mock (回退)
06     """
07     from src.llm.deepseek import DeepSeekLLM
08
09     # 优先尝试 DeepSeek
10     ds = DeepSeekLLM.from_env()
11     if ds.is_available():
12         print(f" [LLM] Using DeepSeek provider
13 (model={ds.config.model})")
14         return ds
15
16     # 回退到规则引擎
17     print(f" [LLM] DeepSeek unavailable, using Mock (rule-
18 based)")
19     return MockLLM(config)
```

DeepSeekLLM 通过 OpenAI-compatible API 调用 deepseek-chat 模型，支持温度、最大 token 数等参数配置。create_llm() 工厂函数自动检测 DEEPSEEK_API_KEY 环境变量，无 Key 时回退 MockLLM。Prompt 模板将偏好解析和投票评价分离，便于独立优化。

DeepSeekLLM 实现：

```

01 class DeepSeekLLM(BaseLLM):
02     def __init__(self, config: LLMConfig | None = None):
03         self.config.provider = "deepseek"
04         self.config.base_url = "https://api.deepseek.com/v1" #
05 OpenAI-compatible
06         self.config.model = "deepseek-chat"
07         self._client = OpenAI(api_key=api_key, base_url=base_url,
08 timeout=30)
09
10     def is_available(self) -> bool:
11         # 检测 DEEPSEEK_API_KEY 环境变量
12         return bool(os.getenv("DEEPSEEK_API_KEY"))

```

环境变量配置:

```

1 # .env (需用户自行创建)
2 DEEPSEEK_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx

```

Prompt 模版分离:

```

1 # 偏好解析 Prompt
2 PREFERENCE_PARSE_PROMPT = """你是一个观影偏好解析专家...
3 只返回 JSON 格式: {"genres": [], "max_duration": 120, "min_rating":
4 7.0, "veto_list": []}"""
5
6 # 投票评分 Prompt
7 VOTE_JUDGE_PROMPT = """你是观影投票助手...
8 返回 JSON: {"score": 8, "verdict": "approve", "reason": "..."}"""

```

6.1.2 接入 Chroma 向量数据库

从 MVP 的内存字典升级到 Chroma 向量数据库。WatchRecord 类将观影记录（影片信息+群体偏好+满意度）转换为文本表示用于向量嵌入。ChromaVectorStore 使用 cosine 相似度检索历史相似决策，Mediator 在生成提案前自动查询并参考历史结果。create_vector_store()工厂函数自动检测 chromadb 是否安装，未安装时回退 MemoryVectorStore（关键词匹配）。

分层存储架构:

```

01 # src/memory/vector_store.py:326-336
02 def create_hierarchical_store(persist_dir: str = "./chroma_db") ->
03 HierarchicalVectorStore:
04     """
05     分层存储架构:
06     - ChromaDB: 持久化存储 (观影历史、影片知识库)
07     - Memory: 临时存储 (当前会话偏好)
08     """
09     store = HierarchicalVectorStore(persist_dir)
10     stats = store.get_stats()
11     print(f" [Memory] Using Hierarchical store (Chroma:
12 {stats.get('chroma_records', 0)}))")
13     return store

```

WatchRecord 文本表示:

```

1 class WatchRecord:
2     def to_text(self) -> str:
3         return f"影片《{self.title}》类型{'/'.join(self.genres)}时长
4 {self.duration}分钟 \
5             群体满意度{self.group_score}分 用户偏好:
6 {self.pref_texts}"

```

语义检索流程:

```

01 class ChromaVectorStore:
02     def query_similar(self, preferences, n_results=3) ->
03 List[dict]:
04     # cosine 相似度检索历史决策
05     results = self._collection.query(
06         query_texts=[query_text],
07         n_results=n_results,
08         include=["metadatas", "distances"]
09     )
10     # 返回相似度加权的参考结果
11     return self._process_results(results)

```

6.1.3 接入真实 MCP

从 MVP 的本地数据库升级到 MCP 协议标准化工具接口, 实现豆瓣/IMDb 等外部数据源接入。douban_mcp.py 定义了 search 和 detail 两个 MCP 工具, 通过 MCP_TOOLS 字典注册。call_tool() 提供统一调用接口, 当 DOUBAN_API_KEY 设置

时调用真实豆瓣 API，否则回退到本地 50 部影片数据库。MCP 协议的优势在于新增工具只需注册新的 MCP Server，无需修改 Agent 核心逻辑。

MCP 工具定义：

```
01 # src/tools/douban_mcp.py
02 MCP_TOOLS = {
03     "douban.search": {
04         "function": search,
05         "description": "按标题/类型/评分/时长搜索影片",
06         "parameters": {
07             "title": {"type": "string"},
08             "genres": {"type": "array", "items": {"type":
09 "string"}}},
10             "max_duration": {"type": "integer"},
11             "min_rating": {"type": "number"}
12         }
13     },
14     "douban.detail": {
15         "function": get_detail,
16         "description": "按 ID 获取影片详情",
17         "parameters": {
18             "movie_id": {"type": "string"}
19         }
20     }
21 }
22
23 def call_tool(tool_name: str, **kwargs) -> str:
24     """统一工具调用接口"""
25     tool = MCP_TOOLS[tool_name]
26     return tool["function"](**kwargs)
```

回退机制：

```
01 def _try_mcp_search(self, genres, max_duration, min_rating,
02 exclude_ids):
03     # 1. 尝试豆瓣 API 爬取
04     results = self._douban_search(genres, max_duration,
05 min_rating)
06
07     # 2. API 失败时回退本地数据库
08     if not results:
09         results = self._local_search(genres, max_duration,
10 min_rating)
11
```

```
12 | # 3. 过滤已否决影片
13 | return [m for m in results if m.movie_id not in exclude_ids]
```

6.1.4 群体规模扩展

从 MVP 的固定 2 人扩展到支持任意 N 人的动态群体，前端和后端均支持无缝扩展。PreferenceAgent 和 MediatorAgent 的设计天然支持多用户，只需传入包含 N 个用户的字典即可。测试覆盖 2 人、4 人和 6 人场景，验证了系统的可扩展性。前端的用户输入表单支持动态添加/删除成员，交互模式下可输入任意数量用户的偏好。

前端动态用户管理：

```
01 | // web/index.html - 动态添加/删除成员
02 | let users = [
03 |   { name: 'Alice', preference: '我喜欢科幻片' },
04 |   { name: 'Bob', preference: '我想看喜剧片' },
05 | ];
06 |
07 | function addUser() {
08 |   users.push({ name: 'User' + (users.length + 1), preference:
09 | '' });
10 |   renderUsers();
11 | }
12 |
13 | function removeUser(idx) {
14 |   if (users.length > 1) {
15 |     users.splice(idx, 1);
16 |     renderUsers();
17 |   }
18 | }
19 |
20 | // 输入验证
21 | if (validUsers.length > 10) {
22 |   showError('最多支持 10 个用户');
23 |   return;
24 | }
```

后端支持：

```

1 # src/api/server.py:179-182
2 if len(req.users) > 10:
3     raise HTTPException(
4         status_code=status.HTTP_400_BAD_REQUEST,
5         detail="最多支持 10 个用户"
6     )
7
8 user_texts = {u.name: u.preference for u in req.users}
9 state = build_consensus_workflow(user_texts, llm=llm,
10 vector_store=vector_store)

```

6.2 未来扩展路线

6.2.1 可扩展架构设计

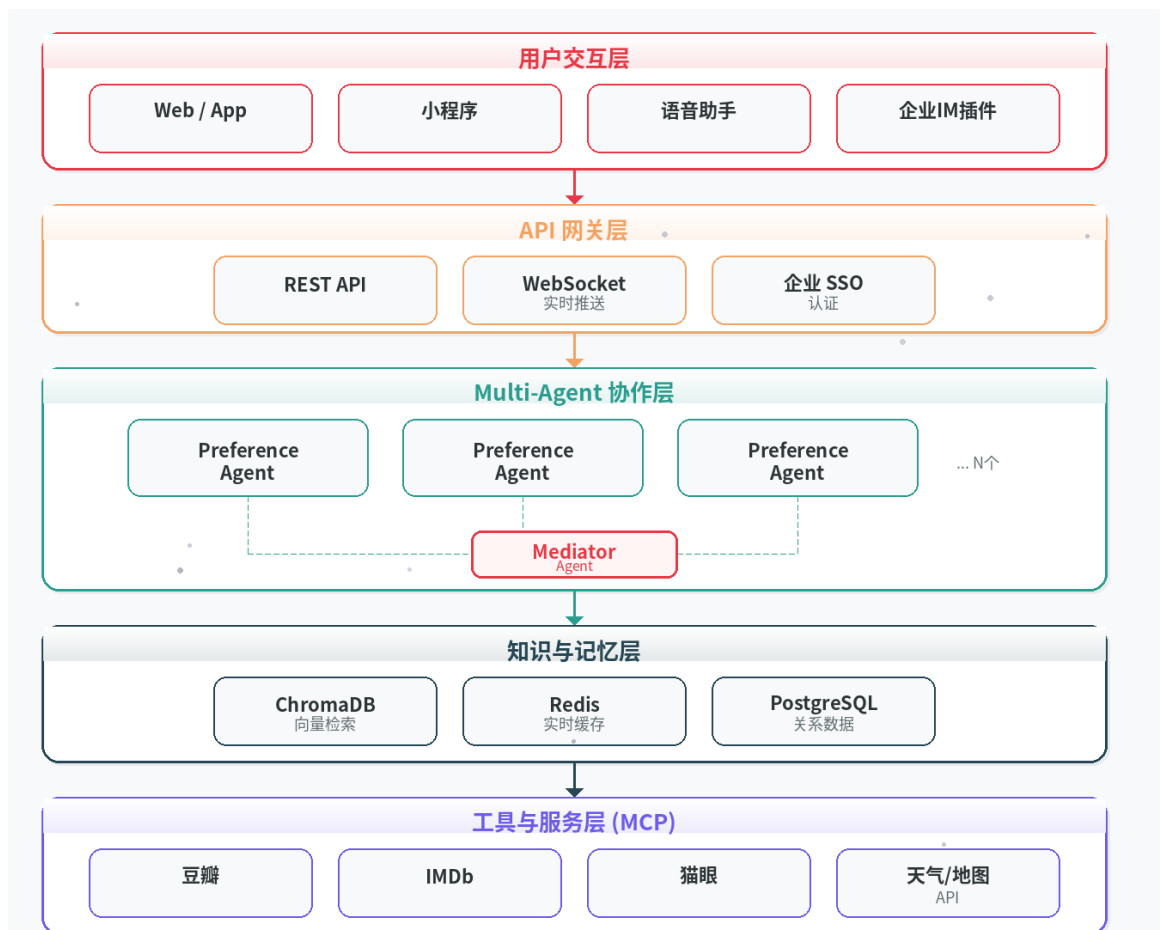


图 6.1 架构分层演进图

插件化 MCP 架构:


```

01 # 未来扩展: MCP 插件注册中心
02 class MCPRegistry:
03     """MCP 工具注册中心, 支持运行时加载新工具"""
04
05     _tools: Dict[str, MCPTool] = {}
06
07     @classmethod
08     def register(cls, name: str, tool: MCPTool):
09         cls._tools[name] = tool
10
11     @classmethod
12     def call(cls, tool_name: str, **kwargs) -> str:
13         if tool_name not in cls._tools:
14             raise ValueError(f"Tool '{tool_name}' not registered")
15         return cls._tools[tool_name].execute(**kwargs)
16
17 # 新增工具只需注册, 无需修改 Agent 逻辑
18 MCPRegistry.register("imdb.search", IMDBTool())
19 MCPRegistry.register("weather.check", WeatherTool())
20 MCPRegistry.register("location.nearby", MapTool())

```

6.2.2 下一阶段计划

阶段	目标	关键特性
Phase 1	移动端适配	响应式 Web / 微信小程序 / React Native
Phase 2	实时协作	WebSocket 实时状态同步 / 多端同步投票
Phase 3	社交功能	观影历史分享 / 群组 PK / 好友推荐
Phase 4	企业版	REST API 开放 / 会议决策插件 / Slack 集成
Phase 5	智能演进	强化学习优化协商策略 / 用户画像自动更新

Phase 1 移动端扩展示例：

```

01 # 新增小程序端 API 兼容
02 @app.post("/api/v2/negotiate")
03 async def negotiate_v2(req: NegotiateRequest):
04     """
05     V2 API: 支持小程序场景
06     - 新增 session_id 支持断点续传
07     - 新增 callback_url 支持异步通知
08     """
09     session_id = req.session_id or generate_session_id()
10     # 存储中间状态到 Redis
11     redis.setex(f"session:{session_id}", 3600,
12 json.dumps(req.dict()))
13     # 异步处理并回调
14     return {"session_id": session_id, "status": "processing"}

```

Phase 2 WebSocket 实时协作：

```

01 # WebSocket 端点：实时推送协商状态
02 @app.websocket("/ws/negotiate/{session_id}")
03 async def websocket_negotiate(websocket, session_id: str):
04     await websocket.accept()
05
06     # 实时推送状态变化
07     async def send_update(state: GraphState):
08         await websocket.send_json({
09             "type": "state_update",
10             "current_state": state.next_state,
11             "votes": [v.dict() for v in state.votes],
12             "log": state.log[-5:] # 最近 5 条日志
13         })
14
15     # 监听投票变化
16     for vote in await collect_votes_streaming(user_ids):
17         await send_update(state)

```

6.3 AI 能力演进路径

当前状态：规则引擎 + DeepSeek LLM 混合模式

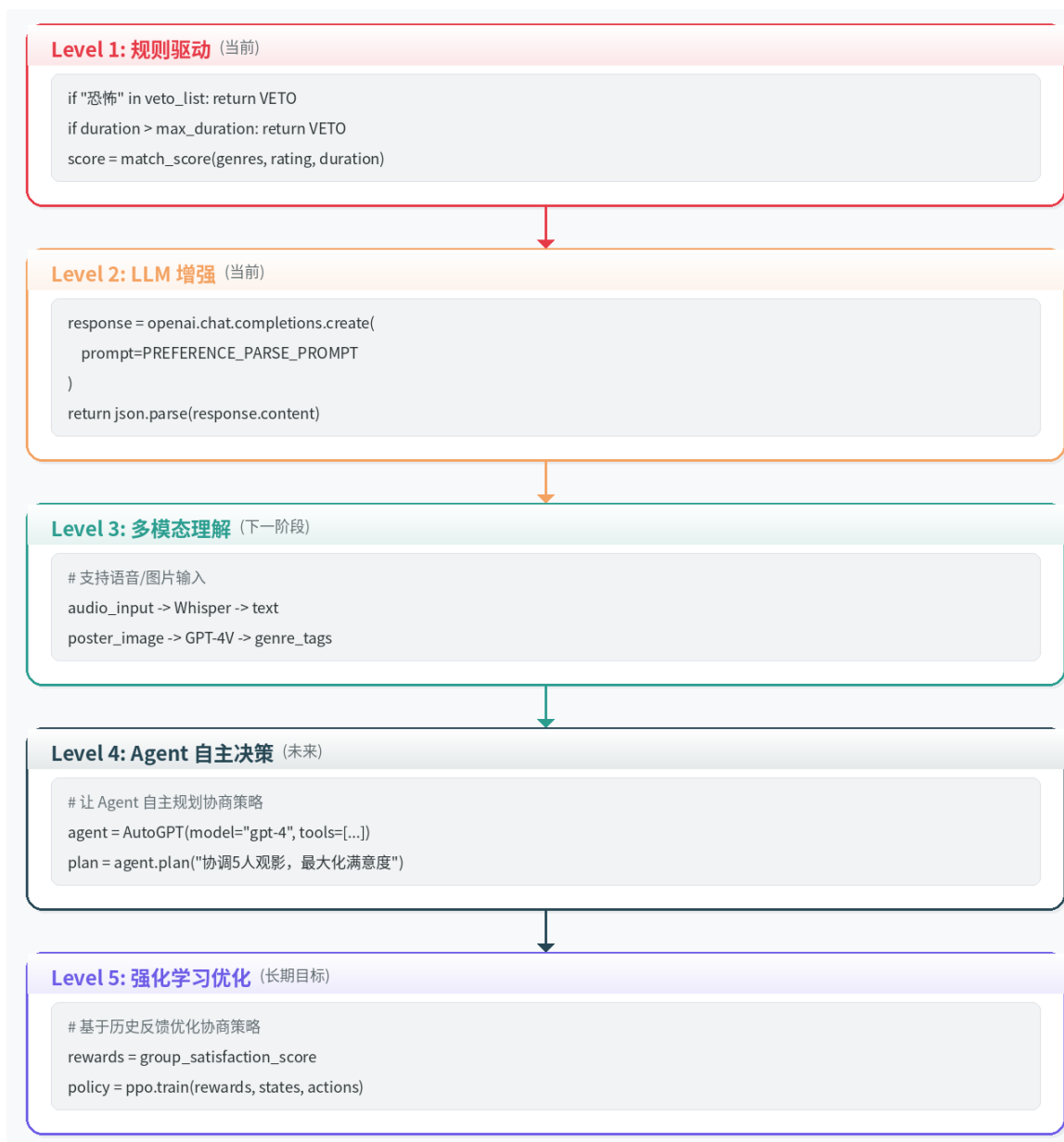


图 6.2 系统扩展路径

Level 3 多模态扩展示例:

```

01 # 语音偏好输入
02 @app.post("/api/voice-preference")
03 async def parse_voice(file: UploadFile):
04     # Whisper 语音转文字
05     audio_data = await file.read()
06     text = whisper.transcribe(audio_data)
07     # 复用现有偏好解析
08     return parse_preference(text)
09
10 # 海报图像识别类型
11 @app.post("/api/image-preference")
12 async def parse_poster(file: UploadFile):
13     image = Image.open(file.file)
14     # GPT-4V 图像理解
15     response = gpt4v.analyze(image,
16                               prompt="识别这张电影海报的类型标签")
17     return extract_genres(response)

```

Level 4 AutoGPT 自主决策示例：

```

01 # 自主协商 Agent
02 class AutonomousMediator:
03     """基于 AutoGPT 的自主协商 Agent"""
04
05     def __init__(self):
06         self.agent = Agent(
07             model="gpt-4",
08             tools=[
09                 search_movies,
10                 query_history,
11                 call_user,
12             ],
13             objectives=[
14                 "最大化群体满意度",
15                 "尊重每个成员的硬约束",
16                 "在 3 轮内达成共识"
17             ]
18         )
19
20     def negotiate(self, users: List[User]) -> ConsensusResult:
21         # Agent 自主决定协商策略
22         plan = self.agent.plan(
23             f"协调 {len(users)} 人观影: {[u.preference for u in
24 users]}"
25         )

```

26 | `return self.agent.execute(plan)`

6.4 部署架构演进

当前：单机部署（开发/演示）

```
1 | python -m uvicorn src.api.server:app --port 8000
```

未来：容器化微服务

```
01 | # docker-compose.yml (未来版本)
02 | services:
03 |     api:
04 |         build: ./api
05 |         ports: ["8000:8000"]
06 |         depends_on: [redis, postgres]
07 |
08 |     worker:
09 |         build: ./worker
10 |         # 处理异步协商任务
11 |
12 |     redis:
13 |         image: redis:alpine
14 |         # 实时状态缓存
15 |
16 |     postgres:
17 |         image: postgres:15
18 |         # 持久化数据存储
19 |
20 |     chroma:
21 |         image: chromadb/chroma
22 |         # 向量数据库
```

可扩展性指标：

指标	当前	Phase4 目标
最大并发用户	10 人/次	1000 人/次
协商响应时间	3-10s	<1s
存储容量	本地磁盘	分布式存储
可用性	单机	99.9%

7 课程总结

7.1 遇到的问题与解决方案

问题一：否定词解析歧义。自然语言中的否定表达具有多种形式，系统需要准确识别用户的"硬约束"和"软偏好"。

问题描述：

- "不要恐怖片" → "恐怖"应加入 veto_list，而非 genres
- "拒绝恐怖和战争" → 两个类型都应加入 veto_list
- "我喜欢科幻片" → "科幻"加入 genres

解决方案：先检测否定前缀（不要/不看/怕/拒绝），再提取类型偏好，对联合表达（"拒绝恐怖和战争"）进行特殊处理分解为两个独立否决项。

```
01 # src/agents/preference_agent.py - 否定词检测逻辑
02 NEGATION_PREFIXES = ["不要", "不看", "怕", "拒绝", "排斥", "强烈拒绝"]
03
04
05 def _declare_rules(self, text: str) -> PreferenceVector:
06     genres, veto_list = [], []
07
08     # 检测否定前缀（前方 8 字符内）
09     for genre in ALL_GENRES:
10         idx = text.find(genre)
11         if idx >= 0:
12             # 检查类型词前方是否有否定词
13             prefix_start = max(0, idx - 8)
14             prefix = text[prefix_start:idx]
15             is_negated = any(neg in prefix for neg in
16 NEGATION_PREFIXES)
17
18             if is_negated:
19                 veto_list.append(genre)
20             else:
21                 genres.append(genre)
22
23     # 联合否决处理："拒绝恐怖和战争" → 分解为两个否决项
```

```

24         for veto in veto_list:
25             veto_texts = text.replace("拒绝", "").replace("和",
26 ",").replace("\、", ",")
27             # 逗号分隔的类型列表单独处理
28

```

问题二：LLM 依赖与离线可用性矛盾。课程 Demo 时 API Key 可能不可用。

问题描述：

- DeepSeek API Key 可能未配置
- 网络隔离环境无法调用外部服务
- Demo 演示时需要稳定输出

解决方案：采用适配器模式+工厂函数，系统启动时自动检测 LLM 可用性，不可用时无缝回退到规则引擎，确保任何环境下均可运行。

```

01 # src/llm/mock.py:207-220 - 工厂函数自动检测
02 def create_llm(config: LLMConfig | None = None) -> BaseLLM:
03     """
04     工厂函数自动检测并创建 LLM provider
05     优先级: DeepSeek (有 API Key) > Mock (回退)
06     """
07     from src.llm.deepseek import DeepSeekLLM
08
09     # 尝试加载 DeepSeek
10     ds = DeepSeekLLM.from_env()
11     if ds.is_available():
12         print(f" [LLM] Using DeepSeek provider
13 (model={ds.config.model})")
14         return ds
15
16     # 无缝回退到规则引擎
17     print(f" [LLM] DeepSeek unavailable, using Mock (rule-
18 based)")
19     return MockLLM(config)
20
21 # DeepSeekLLM 自动检测环境变量
22 class DeepSeekLLM(BaseLLM):
23     @classmethod
24     def from_env(cls) -> "DeepSeekLLM":
25         api_key = os.getenv("DEEPSEEK_API_KEY")
26         return cls(config=LLMConfig(api_key=api_key))

```

```

27
28     def is_available(self) -> bool:
29         return bool(os.getenv("DEEPSEEK_API_KEY"))

```

问题三：前端构建工具安装失败。npm install 在容器环境中超时。解决方案：改用原生 HTML5 + Tailwind CSS CDN 方案，零构建步骤，单文件实现所有功能，通过 Fetch API 直接调用后端。

问题四：向量数据库类型兼容性。ChromaDB 对 metadata 类型有严格限制，不支持嵌套 dict/list。

问题描述：

- user_scores: dict 无法直接存储
- PreferenceVector 包含 list 字段
- 查询结果返回类型异常

解决方案：

```

01 # src/memory/vector_store.py - WatchRecord 类型转换
02 class WatchRecord:
03     def to_dict(self) -> dict:
04         """将观影记录转换为 ChromaDB 兼容的 metadata"""
05         return {
06             "movie_id": self.movie_id,
07             "title": self.title,
08             # dict/list 需转换为字符串
09             "genres": ",".join(self.genres) if
10 isinstance(self.genres, list) else self.genres,
11             "user_scores": json.dumps(self.user_scores) if
12 isinstance(self.user_scores, dict) else self.user_scores,
13             "group_score": self.group_score,
14         }
15
16     @classmethod
17     def from_dict(cls, data: dict) -> "WatchRecord":
18         """从 ChromaDB metadata 恢复对象"""
19         return cls(
20             movie_id=data["movie_id"],
21             title=data["title"],
22             genres=data["genres"].split(",") if
23 isinstance(data["genres"], str) else data["genres"],

```



```
24         user_scores=json.loads(data["user_scores"]) if
25         isinstance(data["user_scores"], str) else data["user_scores"],
26     )
```

7.2 学习收获

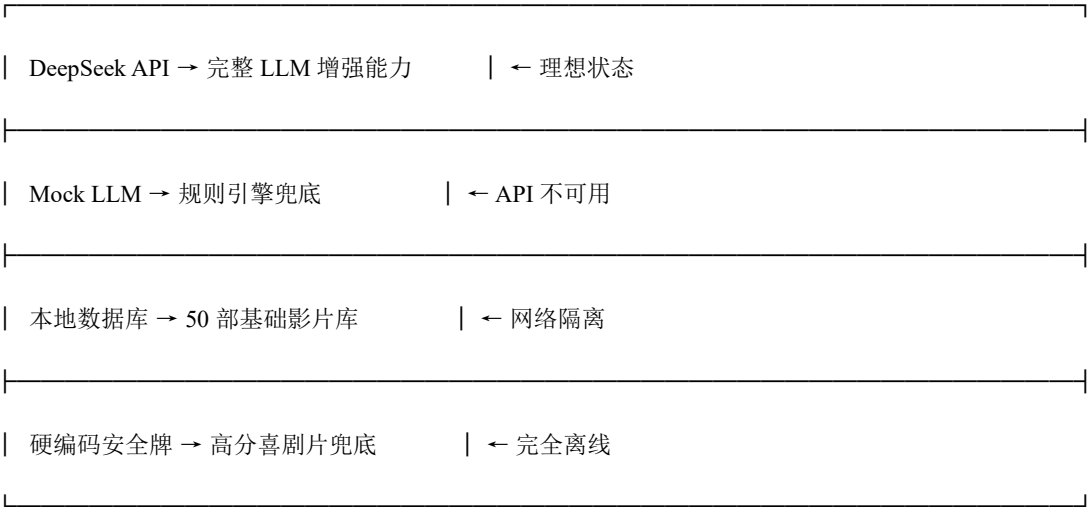
通过本课程项目，深入理解了多智能体协作的核心机制。Preference Agent 的"利益维护"与"灵活妥协"之间的平衡、Mediator Agent 的"冲突检测"与"提案生成"之间的协调，体现了分布式系统中共识算法的精髓。LangGraph 状态机的设计让我认识到，复杂 AI 系统的可控性来自于清晰的状态边界和转换条件。通过本项目深入理解了分布式系统中 共识算法 的核心思想：

Agent 角色	职责	设计模式
Preference Agent	"利益维护者"	代理模式（Proxy）
Mediator Agent	"冲突协调者"	中介者模式（Mediator）

在工程实践层面，掌握了适配器模式、工厂模式等设计模式在 AI 系统中的应用，理解了"优雅降级"（Graceful Degradation）的重要性——系统不应因某个组件失效而完全不可用。测试驱动开发（TDD）的实践让我体会到，良好的测试覆盖不仅保障质量，更是重构和优化的安全网。

优雅降级（Graceful Degradation）实践：

系统可用性保障层级：



7.3 未来改进方向

短期改进：接入真实豆瓣/IMDb API 获取更多影片数据，集成 Chroma 向量数据库实现持久化记忆，接入 DeepSeek API 提升偏好解析准确度。中期改进：支持更多协商维度（导演、演员、年代），实现真正的 LangGraph 迁移（利用 langgraph 库的条件边和并行执行），添加用户反馈闭环（观影后评分更新偏好画像）。长期展望：扩展为多领域群体决策 Agent（餐厅、旅行、购物），接入日历 API 实现时间约束，开发移动端 App 支持扫码组队。



图 7.1 技术路线改进图

具体实现示例：

```

01 # 短期改进: 豆瓣 API 深度集成
02 # src/tools/douban_mcp.py - 扩展搜索维度
03 def search_movies_advanced(
04     title: Optional[str] = None,
05     genres: Optional[List[str]] = None,
06     director: Optional[str] = None,      # 新增: 导演筛选
07     actor: Optional[str] = None,        # 新增: 演员筛选
08     year_range: Optional[Tuple[int, int]] = None,  # 新增: 年代筛选
09     max_duration: Optional[int] = None,
10     min_rating: Optional[float] = None,
11 ) -> List[Movie]:
12     """高级搜索接口, 支持多维度筛选"""
13     pass
14
15 # 中期改进: 用户反馈闭环
16 # src/agents/preference_agent.py - 偏好画像更新
17 class PreferenceProfile:
18     """用户偏好画像, 动态更新"""
19     def __init__(self, user_name: str):
20         self.user_name = user_name
21         self.genre_scores = defaultdict(float)  # 类型 → 平均评分
22         self.avg_duration = 0
23         self.avg_rating = 0
24
25     def update_from_watch(self, watch_record: WatchRecord):
26         """根据观影历史更新偏好画像"""
27         score = watch_record.user_scores.get(self.user_name, 5)
28         for genre in watch_record.genres:
29             self.genre_scores[genre] = (self.genre_scores[genre] +
30 score) / 2
31
32 # 长期改进: 多领域扩展
33 class ConsensusDomain(Enum):
34     MOVIE = "movie"      # 观影决策
35     RESTAURANT = "restaurant" # 餐厅决策
36     TRAVEL = "travel"    # 旅行决策
37     SHOPPING = "shopping" # 购物决策
38
39 class UniversalMediator:
40     """通用协商中介者, 支持多领域"""
41     def __init__(self, domain: ConsensusDomain):
42         self.domain = domain
43         self.prompt_template = self._load_template(domain)
44

```

性能优化方向：

```
01 # 缓存优化：减少重复 LLM 调用
02 class LLMCache:
03     """LLM 响应缓存"""
04     def __init__(self, redis_client):
05         self._cache = redis_client
06
07     def get(self, key: str) -> Optional[LLMResponse]:
08         cached = self._cache.get(f"llm:{hash(key)}")
09         return LLMResponse(**json.loads(cached)) if cached else
10 None
11
12     def set(self, key: str, response: LLMResponse, ttl: int =
13 3600):
14         self._cache.setex(f"llm:{hash(key)}", ttl,
15 json.dumps(response.dict()))
16
17 # 并行优化：批量查询 Chroma
18 class BatchVectorStore:
19     """批量向量查询优化"""
20     async def query_batch(self, queries: List[str], n: int = 3):
21         """并行执行多个向量查询"""
22         tasks = [self.query_similar(q, n) for q in queries]
23         return await asyncio.gather(*tasks)
```